

第六章 主存管理

提纲

6.1 存储管理功能

6.2 内存资源管理

6.3 存储管理方式

提纲

6.1 存储管理功能

6.2 内存资源管理

6.3 存储管理方式

6.1 存储管理功能

- 存储分配和去配
 - 分配去配对象
 - 内存、外存(相同方法)
 - 分配去配时刻
 - 进程创建、撤销、交换、长度变化(栈溢出等)
 - 分配表：记录已经分配的区域
 - 空闲表：记录尚未分配的区域

6.1 存储管理功能(Cont.)

- 存储**共享**
 - 目的：节省内存、相互通讯
 - 内容：代码、数据
- 存储**保护**
 - 防止地址越界
 - 防止操作越权

6.1 存储管理功能(Cont.)

■ 存储扩充

- 内存、外存结合，虚拟存储体系
- 速度接近内存，容量相当外存

■ 地址映射

- 逻辑地址=>物理地址
- 硬件支持
 - 基址寄存器(base)、限长寄存器(limit)、快表;
- 方式
 - 固定地址映射
 - 静态地址映射
 - 动态地址映射

6.1 存储管理功能(Cont.)

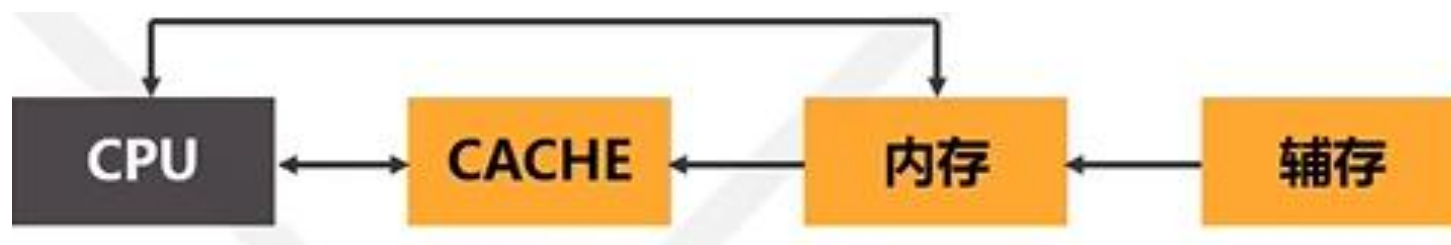
- 固定地址映射
 - 编译时确定逻辑地址和物理地址映射关系
 - 特点
 - 程序加载时必须放在指定的内存区域
 - 容易产生地址冲突，导致运行失败
- 静态地址映射
 - 程序装入时由操作系统完成逻辑地址到物理地址的映射
 - 如在桌面上双击程序的图标，会将其由硬盘转入内存
 - 如在命令行上输入程序名字，回车时就是程序装入时
 - 物理地址=逻辑地址+装入地址
 - 特点：
 - 程序运行之前确定映射关系
 - 程序装入后不能移动
 - 如果移动必须放回原来位置
 - 程序占用连续内存空间

6.1 存储管理功能(Cont.)

- 动态地址映射
 - 在程序**执行过程中**完成逻辑地址到物理地址的映射
 - 例：MOV AX [500]; 访问500单元时执行地址转换
 - 动态重定位
 - 在程序执行期间每次访问内存之前进行重定位
- 缺点
 - 硬件支持（**MMU：内存管理单元**）

6.1 存储管理功能(Cont.)

- 实际存储体系
 - 三级存储体系



- Cache (快, 小, 贵) + 内存 (适中) + 辅存 (慢, 大, 廉)
- 基本原理
 - 当内存太小不够用时, 用辅存来支持内存
 - 暂时不运行的程序换出到辅存上, 必要时再换入内存

6.1 存储管理功能(Cont.)

- 存储器访问的局部性原理
 - 时间局部性
 - 正在访问的存储单元在不久的将来还会被访问
 - 空间局部性
 - 较短时间内，访问的存储地址序列分布在地址空间的较小范围内

提纲

6.1 存储管理功能

6.2 内存资源管理

6.3 存储管理方式

6.4 外存空间管理

6.5 虚拟存储系统

6.2 内存资源管理

- 内存分区

- 分区时刻

- 静态分区：系统初始化时分；
 - 动态分区：申请时分，即在系统运行过程中划分

- 分区大小

- 等长分区
 - 异长分区

- 通常作法

- 静态+等长（页式、段页式）
 - 动态+异长（段式、界地址）。

内存分配

- 静态等长分区的分配
 - 字位映象图
 - 空闲页面表
 - 空闲页面链
- 动态异长分区的分配
 - 空闲区域表
 - 最先适应 (First Fit)
 - 下次适应法 (Next Fit)
 - 最佳适应 (Best Fit)
 - 最坏适应 (Worst Fit)

静态等长分区的分配

1 位示图 (bit map)

用一个bit代表一页状态，0表空闲，1表占用。（多单元）

1	0	0	...	1	...	1	0
---	---	---	-----	---	-----	---	---

第
0
页

第
1
页

第
2
页

...

第
n
页

...

第
m
页

分配：自头寻找第一个为0的位，改为1，返回页号；

去配：找到页后，将页号对应的位(bit)置为0。

静态等长分区的分配 (Cont.)

2 空闲页面表

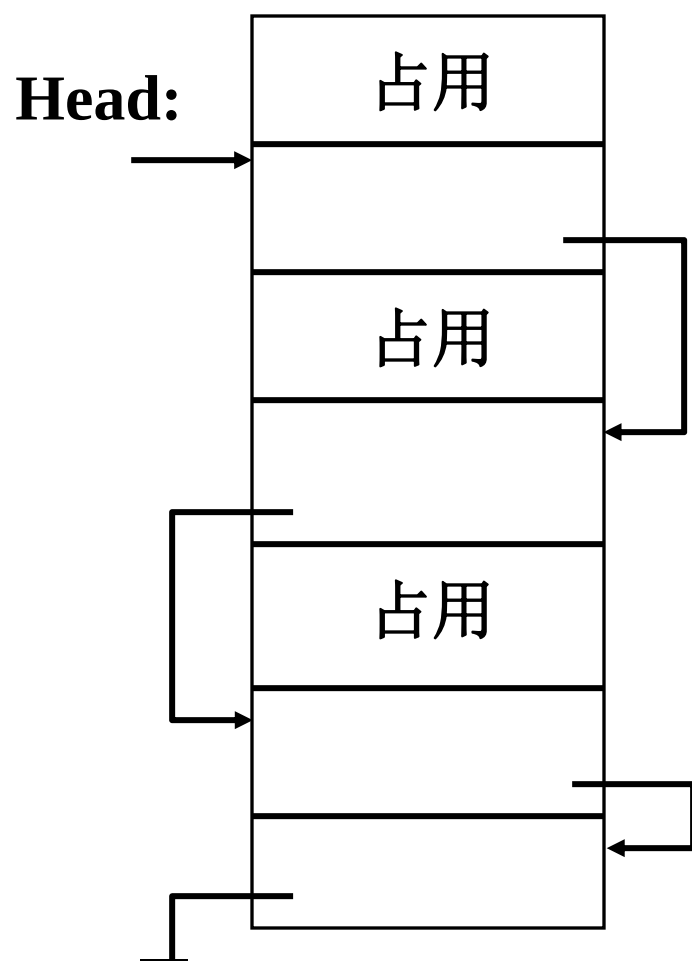
首页号	空页数
...	...
120	4
...	...

特点：若干个连续的空闲页面作为一组登记在空闲页面表中。

...
占用
120页
121页
122页
123页
占用
...

静态等长分区的分配 (Cont.)

3 空闲页面链



特点：将所有空闲页面连成一个空闲页面链。

优点：节省空间。

缺点：对于外存页面的分配和去配均需执行一次数据传输，速度较慢。

动态异长分区的分配

数据结构：

空闲区首址	空闲区长度
...	...
2500	1500
...	...

初始时一个连续空闲区。

长度=0为表尾。

空闲区域表：

记录当前未被进程占用的空闲区域。

Criteria:

尽量使空闲区域连续。

动态异长分区的分配 (Cont.)

- 最先适应 (First Fit)
- 下次适应 (Next Fit)
- 最佳适应 (Best Fit)
- 最坏适应 (Worst Fit)

最先适应算法 (First Fit)

空闲区首址	空闲区长度
128	64
256	32
1024	256
	0
...	...

空闲区：首址递增排列；
申请：取满足申请长度要求
且起始地址最小的空闲
区域；
优点：尽量使用低地址空间，
高区保持大空闲区域。
缺点：可能分割大空闲区。
Eg. 申请32将分割第
一个区域。

下次适应算法 (Next Fit)

空闲区首址	空闲区长度
128	64
256	32
1024	256
	0
...	...

空闲区：首址递增排列；

申请：自上次分配空闲区域的下一个位置开始，
选取第一个满足区域；

优点：减少查找空闲区域所花费的时间开销，并使空闲区域分布更均匀。

缺点：可能分割较大空闲区域。

最佳适应算法 (Best Fit)

空闲区首址	空闲区长度
128	64
256	32
1024	256
	0
...	...

空闲区：首址递增排列；

申请：取最小可满足区域；

优点：尽量使用小空闲区，
保持大空闲区。

缺点：可能形成碎片
(fragment)。

Eg. 申请30将留下长度为2的空闲区。

最坏适应算法 (Worst Fit)

空闲区首址	空闲区长度
128	64
256	32
1024	256
	0
...	...

空闲区：首址递增排列；

申请：取最大可满足区域；

优点：防止形成碎片。

缺点：分割大空闲区域。

UNIX存储分配—FF

First Fit算法例子

m_addr	m_size
100k	30k
200k	90k
300k	60k
	0

申请50k
→
返回200k

m_addr	m_size
100k	30k
250k	40k
300k	60k
	0

First Fit算法例子

m_addr	m_size
100k	30k
250k	40k
300k	60k
	0

申请40k
 →
 返回250k

m_addr	m_size
100k	30k
300k	60k
	0

First Fit算法例子(Cont.)

m_addr	m_size
100k	30k
300k	60k
	0

释放
→
(200k, 50k)

m_addr	m_size
100k	30k
200k	50k
300k	60k
	0

First Fit算法例子(Cont.)

m_addr	m_size
100k	30k
200k	50k
300k	60k
	0

释放
→
(250k, 40k)

m_addr	m_size
100k	30k
200k	90k
300k	60k
	0

First Fit算法例子(Cont.)

m_addr	m_size
100k	30k
200k	90k
300k	60k
	0

释放

(150k, 50k)

m_addr	m_size
100k	30k
150k	140k
300k	60k
	0

First Fit算法例子(Cont.)

m_addr	m_size
100k	30k
150k	140k
300k	60k
	0

释放

(130k, 20k)

m_addr	m_size
100k	190k
300k	60k
	0

UNIX存储分配--FF

```
struct map    //存储资源表结构
{
    char *m_size;
    char *m_addr;
};
```

```
struct map coremap[CMAPSIZE];    //内存空闲区域表
struct map swapmap[SMAPSIZE];    //外存空闲区域表
```

```
define CMAPSIZE 100
define SMAPSIZE 100
```

```
malloc(mp,size) //存储分配函数
struct map, *mp;
{   register int a;
    register struct map *bp;
    for(bp = mp; bp->m_size; bp++){
        if (bp->m_size >= size) {
            a=bp->m_addr; //找到分配区域的首地址
            bp->m_addr =+ size;
            if ((bp->m_size -= size) == 0)
                do {
                    bp++;
                    (bp-1)->m_addr = bp->m_addr;
                }while((bp-1)->m_size = bp->m_size);
            //指针后移，直至到达表尾结束
            return(a); //返回分配区域的首地址
        }
    }
    return(0); }
```

mfree(mp,size,aa) //存储释放函数， aa为释放区域起始地址

struct map *map;

{ register struct map bp;

register int t,a;

a = aa;

for(bp=mp; bp->m_addr<=a && bp->m_size !=0; bp++);

if(bp>mp && (bp-1)->m_addr+(bp-1)->m_size == a) { //与前合并

(bp-1)->m_size =+ size;

if (a+size == bp->m_addr){ //前后合并

(bp-1)->m_size =+ bp->m_size;

while (bp->m_size) {

bp++;

(bp-1)->m_addr = bp->m_addr;

(bp-1)->m_size = bp->m_size;

}

}

```
}else{  
    if (a+size == bp->m_addr && bp->m_size) {    //与后合并  
        bp->m_addr -= size;  
        bp->m_size += size;  
    } else if (size) do {    //无合并  
        t = bp->m_addr;  
        bp->m_addr = a;  
        a = t;    //a与bp->m_addr交换  
        t = bp->m_size;  
        bp->m_size = size;  
        bp++;    //size与bp->m_size交换  
    }while (size = t);  
  
}  
  
}
```


例:

某基于动态分区存储管理的计算机，其主存容量为55MB（初始为空闲），采用最佳适配（Best Fit）算法，分配和释放的顺序为：分配15MB、分配30MB、释放15MB、分配8MB、分配6MB，此时主存中最大空闲分区的大小是_____.

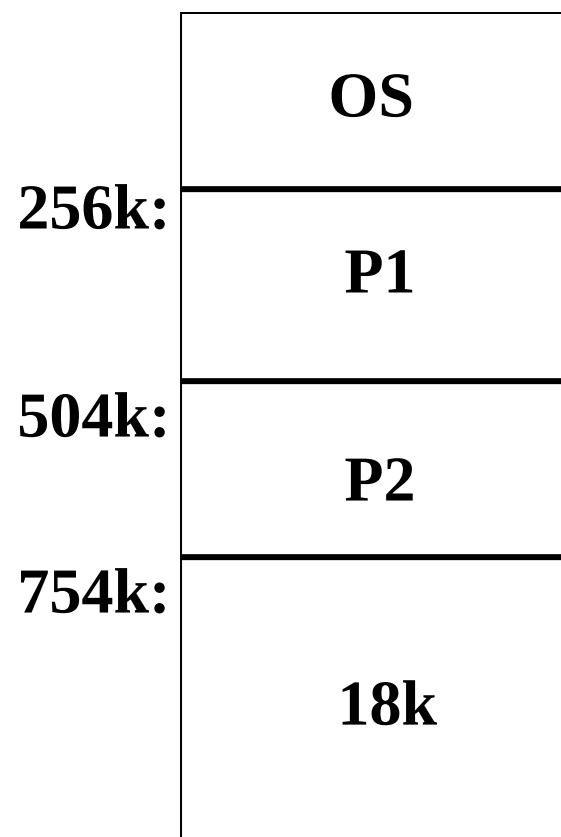
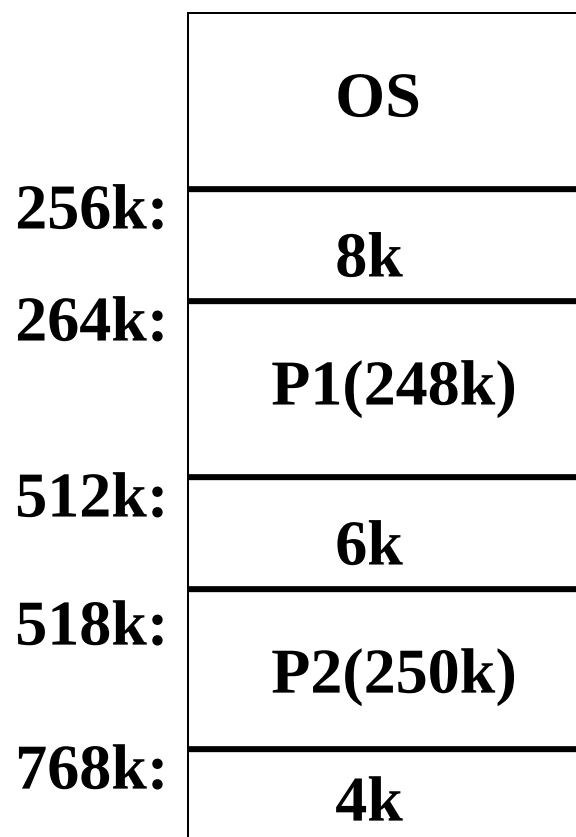
55MB	15MB	15MB	15MB	15MB	6MB
					9MB
	40MB	30MB	30MB	30MB	30MB
		10MB	10MB	8MB	8MB
				2MB	2MB
初始	分配15MB	分配30MB	释放15MB	分配8M	分配6MB

6.2.3 碎片处理

- 碎片
 - 动态异常分区存储分配可能形成很小的空闲区域，称为碎片(fragment)
 - 如果碎片很多，将造成严重的存储资源浪费
 - 解决方法-紧凑(compact ion)
 - 移动所有的占有区域，以使所有的空闲区域连成一片
 - 缺点：开销大

6.2.3 碎片处理(Cont.)

紧凑：移动占用区域，使所有空闲区域连成一片（开销很大）。



6.2.3 碎片处理(Cont.)

- 紧凑的时机
 - 系统找不到足够大的空闲区时
 - 空闲区的管理复杂
 - 定期
 - 空闲区的管理复杂
- 缺点
 - 消耗系统资源
 - 使得所有进程暂停，系统紧凑)

提纲

6.1 存储管理功能

6.2 内存资源管理

6.3 存储管理方式

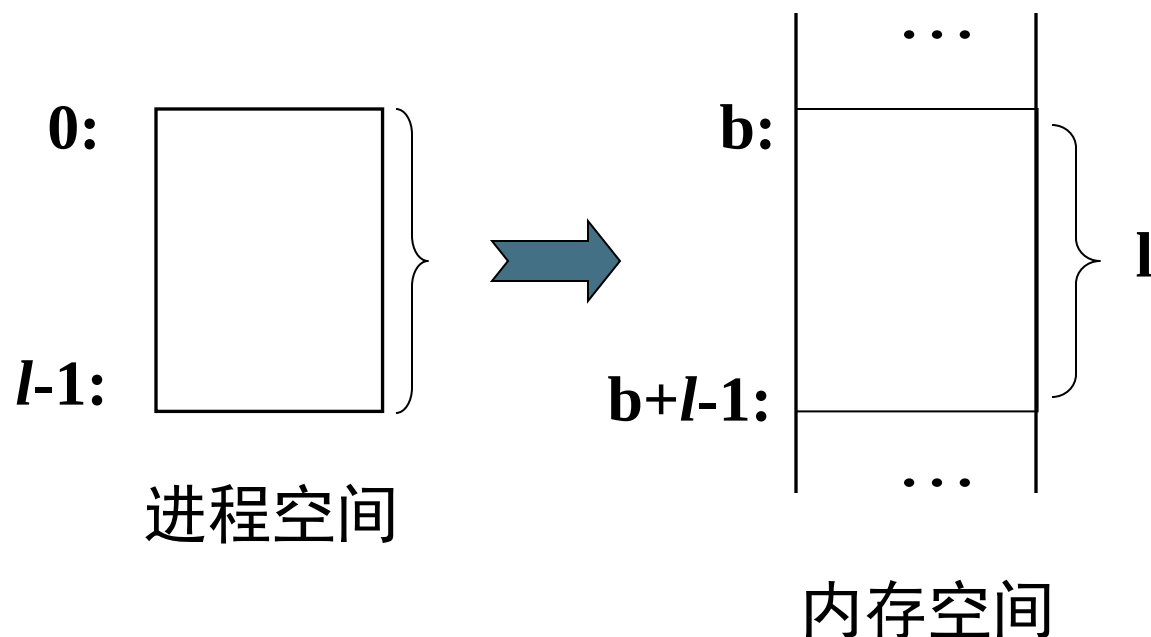
6.3 存储管理方式

- 界地址管理方式(一维地址)
- 页式管理方式(一维地址)
- 段式管理方式(二维地址)
- 段页式管理方式(二维地址)

6.3.1 界地址管理方式

■ 基本原理

1. 内存空间划分：动态异长；
2. 进程空间划分：一个进程一个区域，逻辑地址 $0 \sim l-1$
3. 进程空间与内存空间对应关系：



6.3.1 界地址管理方式(Cont.)

4. 所需表目

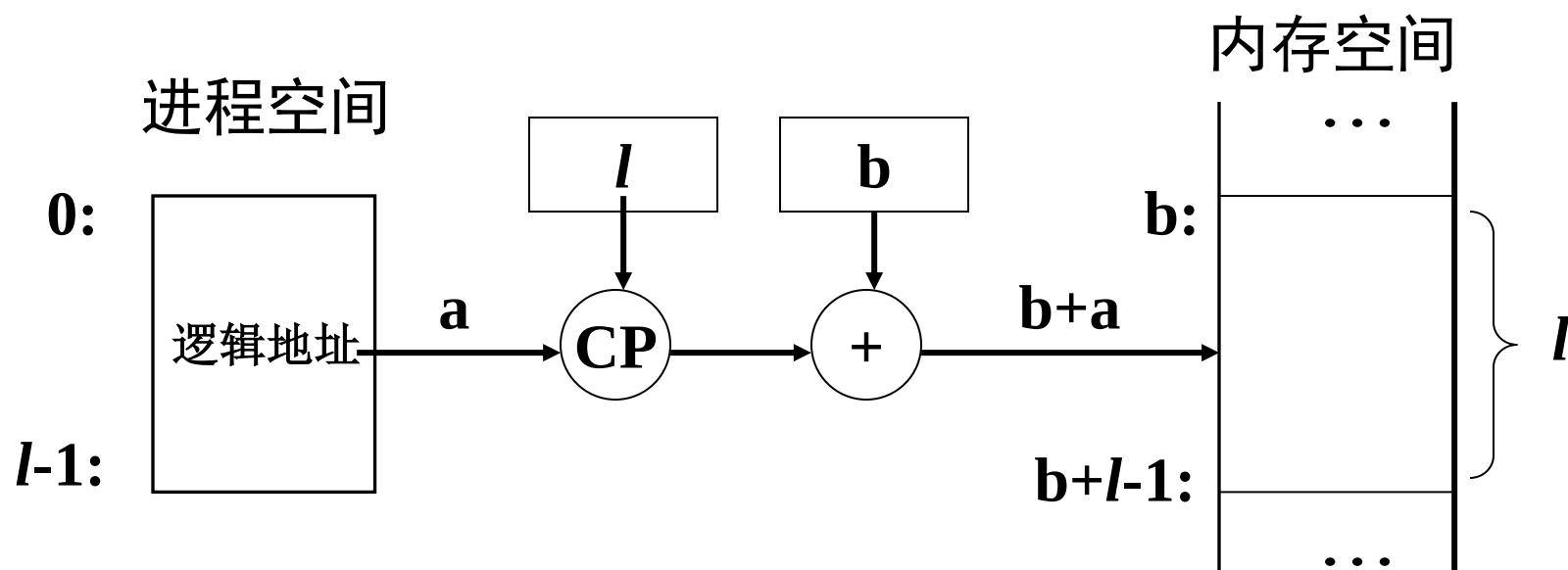
- (1) 内存分配表：记录内存中所有已经被分配的区域
- (2) 空闲区域表：记录内存中尚未分配的区域

5. 所需寄存器

- (1) 基址(首址)寄存器：保存正在运行进程起始地址
- (2) 限长寄存器：保存运行进程长度

6. 地址映射：将程序产生的逻辑地址转换为内存中的物理地址

6.3.1 界地址管理方式(Cont.)



- 步骤：(1) 由程序确定逻辑地址 a ；
- (2) a 与 l 比较判断是否越界，
不满足： $0 \leq a \leq l-1$ ，越界；
- (3) a 与 b 相加得到物理地址。

6.3.1 界地址管理方式(Cont.)

- 双对界：允许一个进程在内存中占有2个连续的区域。

代码(I空间)：一对界

b1

<i>l1</i>

数据(D空间)：一对界

b2

<i>l2</i>

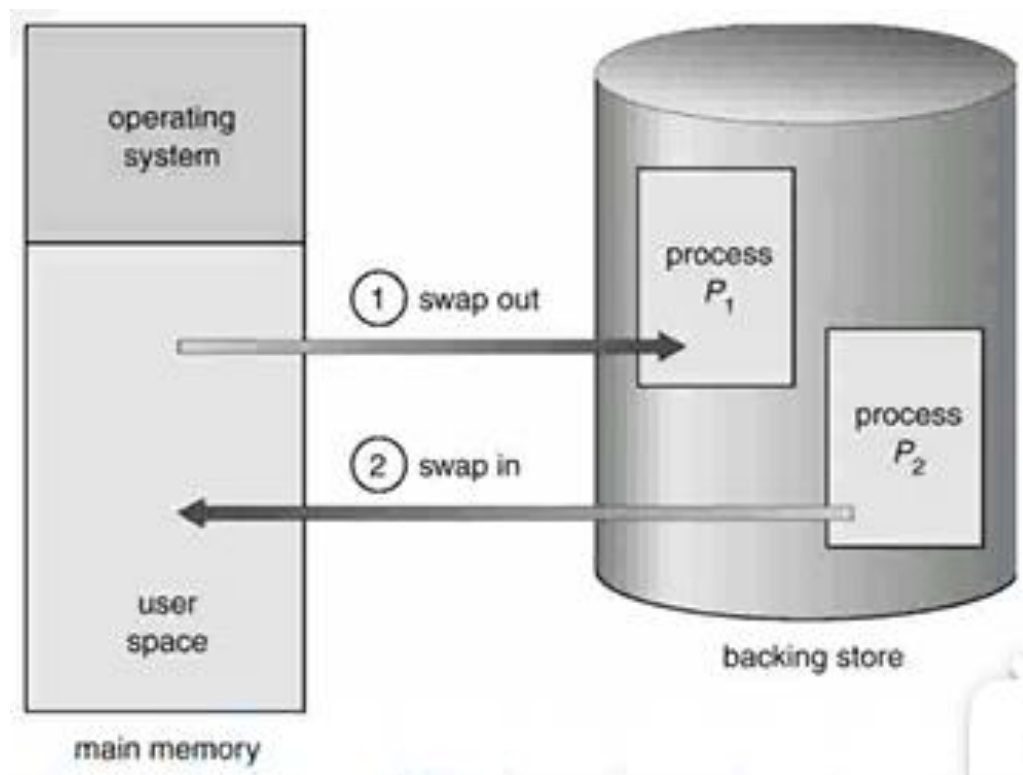
实现：硬件需提供两对寄存器。硬件能识别逻辑地址的所属空间

注：代码区域可以被多个进程所共享，数据区域则为进程所独享

交换技术(swapping)

■ 原理

- 内存不够时把进程写到磁盘（换出/Swap Out）
- 当进程要运行时重新写回内存（换入/Swap In）



交换技术(swapping)

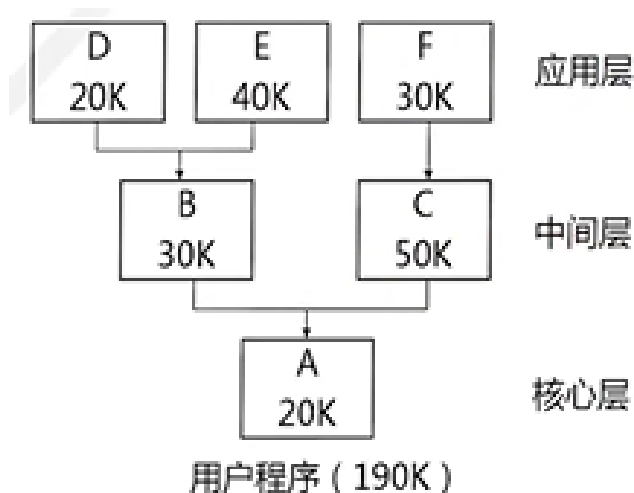
- 缺点
 - 换入换出增加CPU开销（IO操作）
 - 交换单位太大（整个进程）
- 需要考虑的问题
 - 外存交换空间的管理方法
 - 程序换入时的地址重定位
- 例：UNIX 交换进程sched（#0）

重定位

- ◆ 可重定位程序：被换出的进程再次运行之前必须重新装入内存，而再次进入内存时的存放位置与换出之前通常不同，这就要求程序编址与内存存放位置无关，这种程序称为可重定位程序。
- ◆ 浮动程序：由0起始编址的程序都满足重定位要求，这样的程序也称为浮动程序。
- ◆ 动态重定位：在进程运行时进行
- ◆ 静态重定位：在进程运行前编译时或装入时进行

覆盖技术

- 目的
 - 在较小的内存空间中运行较大的程序
- 内存分区
 - 常驻区：被全局代码和数据占用，可划分多个
 - 覆盖区：能被多个段共同（覆盖），可划分多个

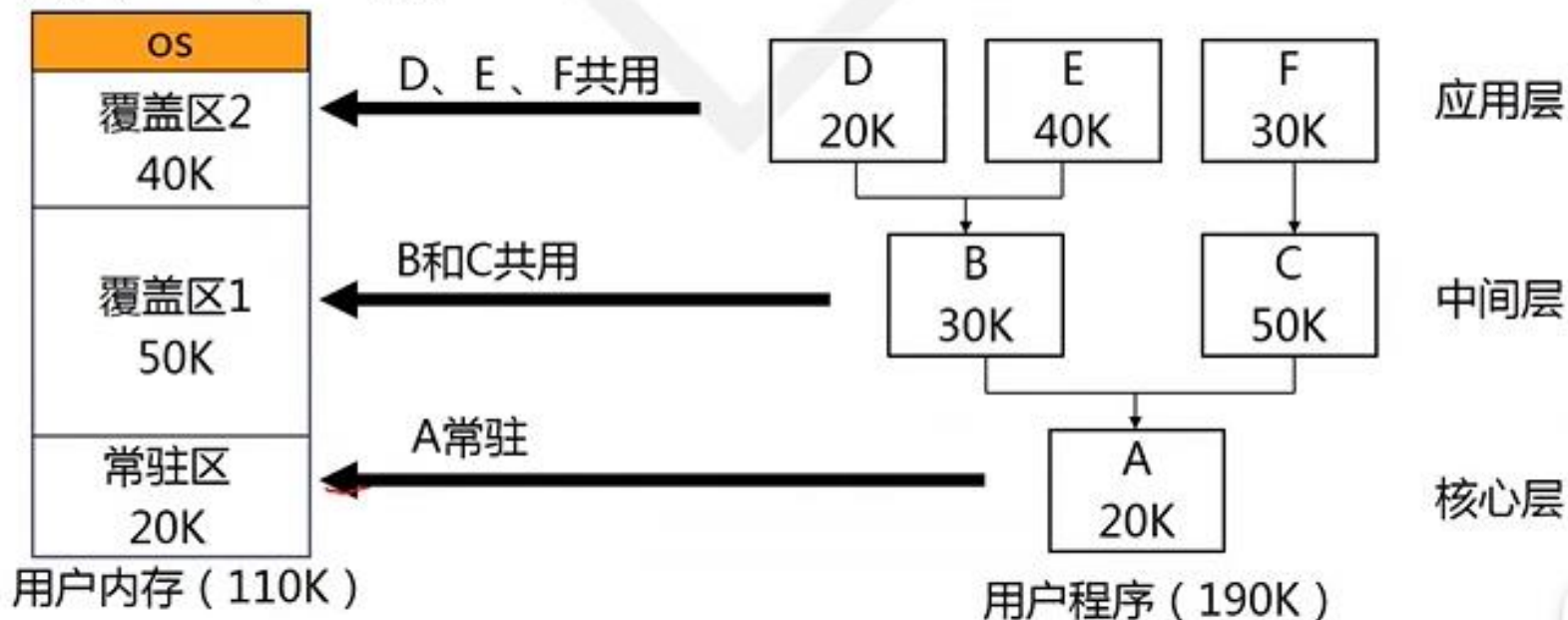


覆盖技术(Cont.)

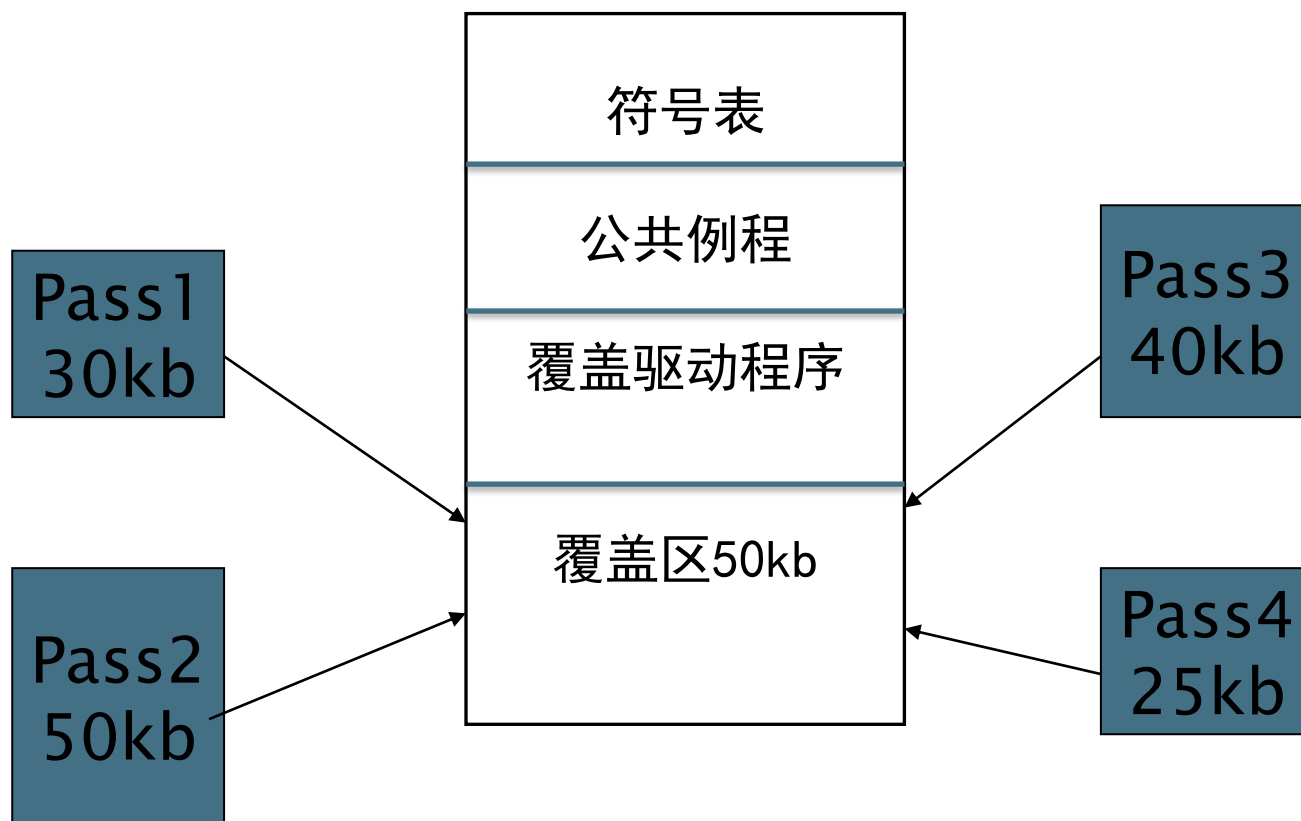
覆盖——Overlay

覆盖的例子

- 内存 (110K) : 一个常驻区, 两个覆盖区
- 程序 (190K) : 6个段



覆盖技术(Cont.)



覆盖技术

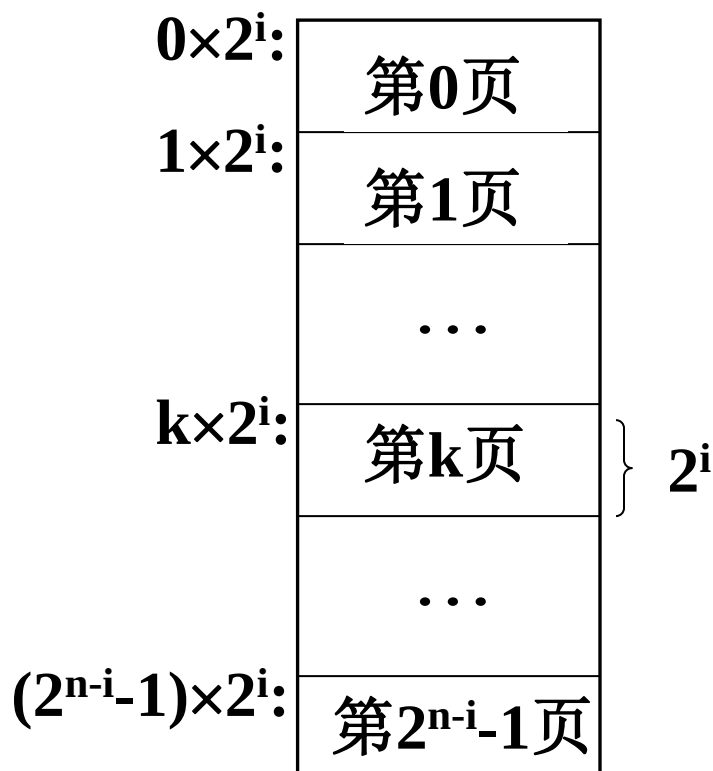
- 缺点

- 程序员划分程序模块并确定覆盖关系
- 程序执行时间长：从外存装入内存耗时

6.3.2 分页式存储管理(paging)

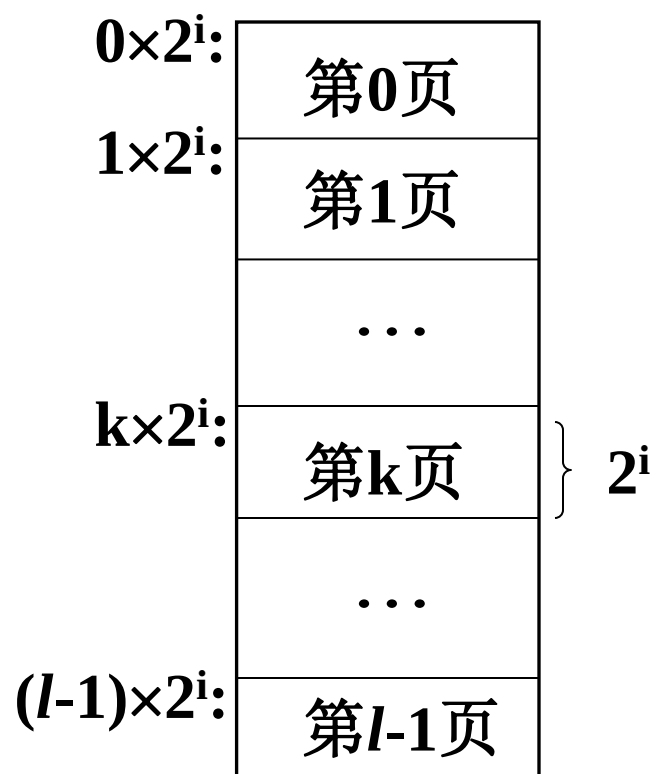
■ 基本原理

1. 内存空间划分：静态等长， 2^i ，称为一个页框(frame)。



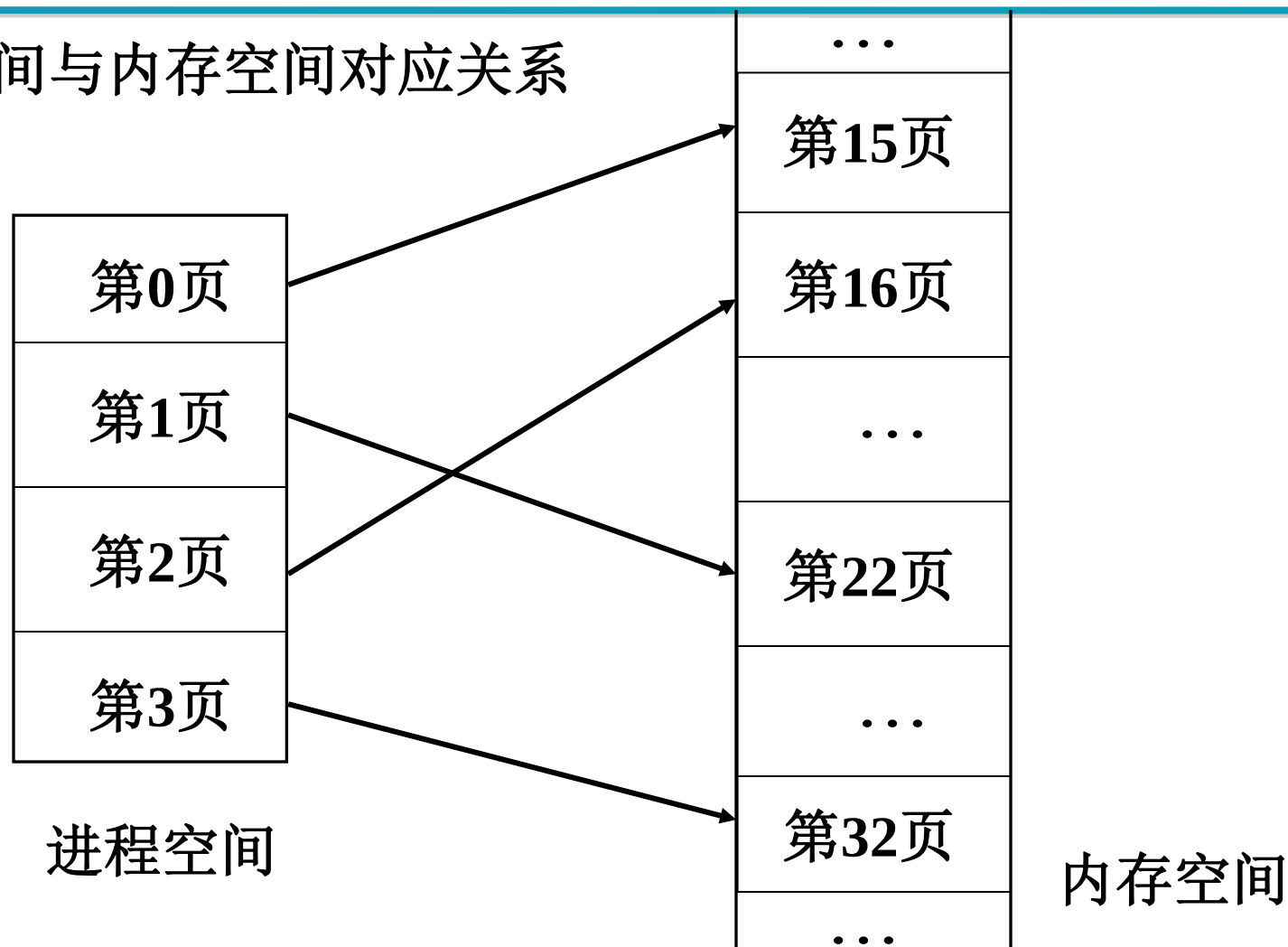
$$\begin{aligned}
 \text{物理地址} &= \text{页框首址} + \text{页内地址} \\
 &= \text{页框号} \times 2^i + \text{页内地址} \\
 &= \underbrace{\text{页框号}}_{n-i \text{ 位}} \underbrace{\text{页内地址}}_{i \text{ 位}}
 \end{aligned}$$

2. 进程空间划分：静态等长， 2^i ，称为一个页面。



$$\begin{aligned}
 \text{逻辑地址} &= \text{逻辑页首址} + \text{页内地址} \\
 &= \text{逻辑页号} \times 2^i + \text{页内地址} \\
 &= \boxed{\text{逻辑页号}} \underbrace{\boxed{\text{页内地址}}}_{i\text{位}}
 \end{aligned}$$

3. 进程空间与内存空间对应关系



4. 所需表目:

(1)页表, 每个进程一个

逻辑页号:	页框号
0	15
1	22
2	16
3	32

(2)总页表: 系统一个, 记录内存的分配和使用状态

5. 所需寄存器

(1) 页表首址寄存器:

b	系统一个
----------	------

(2) 页表长度寄存器:

l	系统一个
----------	------

(3) 快表(TLB): 系统一组:

逻辑页号	页框号
...	...
p	f
...	...

快表

- 快表（TLB）：一组联想寄存器，用于保存正在运行进程的页表中的部分表项。
 - 页表长度一般大于快表的长度；
 - 通常只将正在访问的页面所对应的项目记录于快表中；
 - 访问一个页面时，由逻辑页号先在快表中进行查找，如果找到则根据页框号和页内地址形成物理地址；如果没找到，则由逻辑页号在页表中查找，并将查找的页框号和页号送入快表，如快表满则淘汰。

内存的有效访问时间

- 快表命中率：访问页号在快表的次数与总访问次数之比
 - 根据程序的行为特性
 - 快表长度达到16，命中率很高，再增加不明显
- (Effective Access Time) 有效访问时间
 - $EAT = \text{快表命中率} \times (\text{快表访问时间} + \text{内存访问时间}) + (1 - \text{快表命中率}) \times (\text{快表访问时间} + 2 \times \text{内存访问时间})$ ns
 - $98\% \times (20 + 100) + 2\% \times (20 + 200) \text{ ns} = 122 \text{ ns}$

6. 地址映射 $\sigma: (p,d) \rightarrow (f,d) \cup \{\Omega\}$

逻辑地址 $(p,d) \Rightarrow$ 物理地址 (f,d)

(1) 由程序确定逻辑地址 (p,d) ;

(2) 由 p 查快表得页框号 f ;

如查不到:

(a) 由 p 与 l 比较, 判别是否越界:

不满足: $0 \leq p \leq l-1$, 越界;

(b) 由 p 和 b 查页表得 f ;

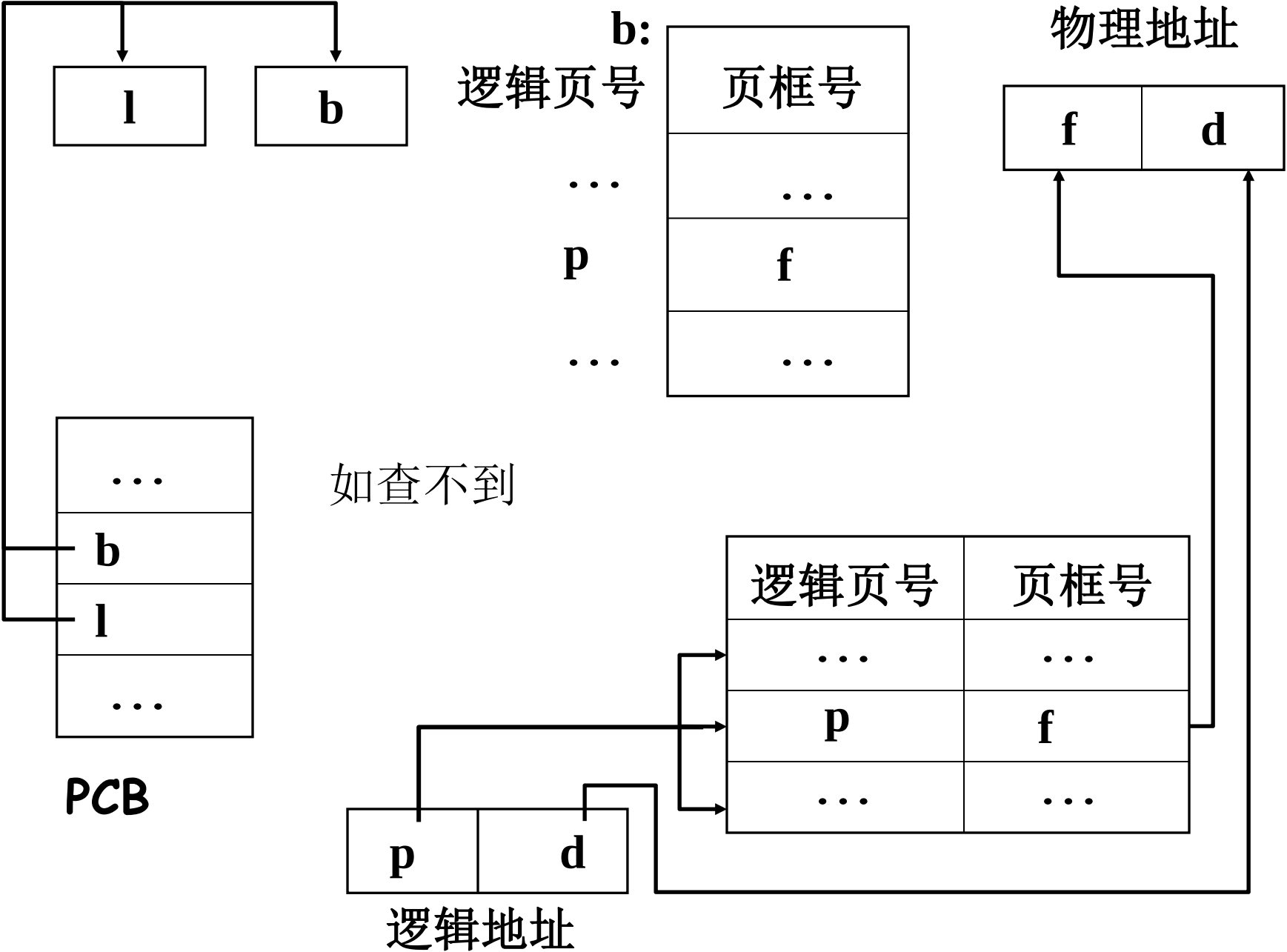
(c) **parbegin**

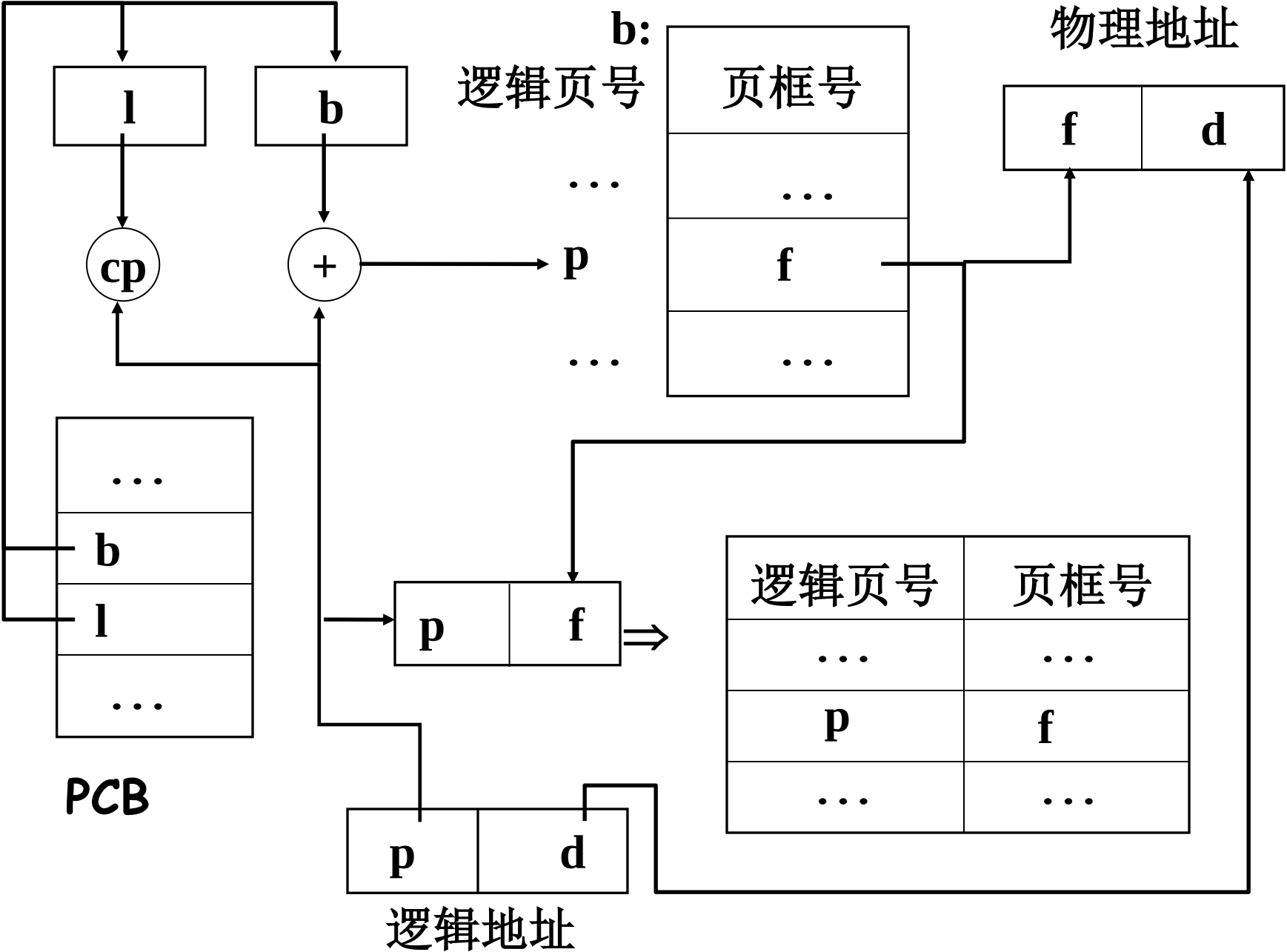
f 与 d 合并得物理地址

$(p,f) \Rightarrow$ 快表, 如满淘汰一个;

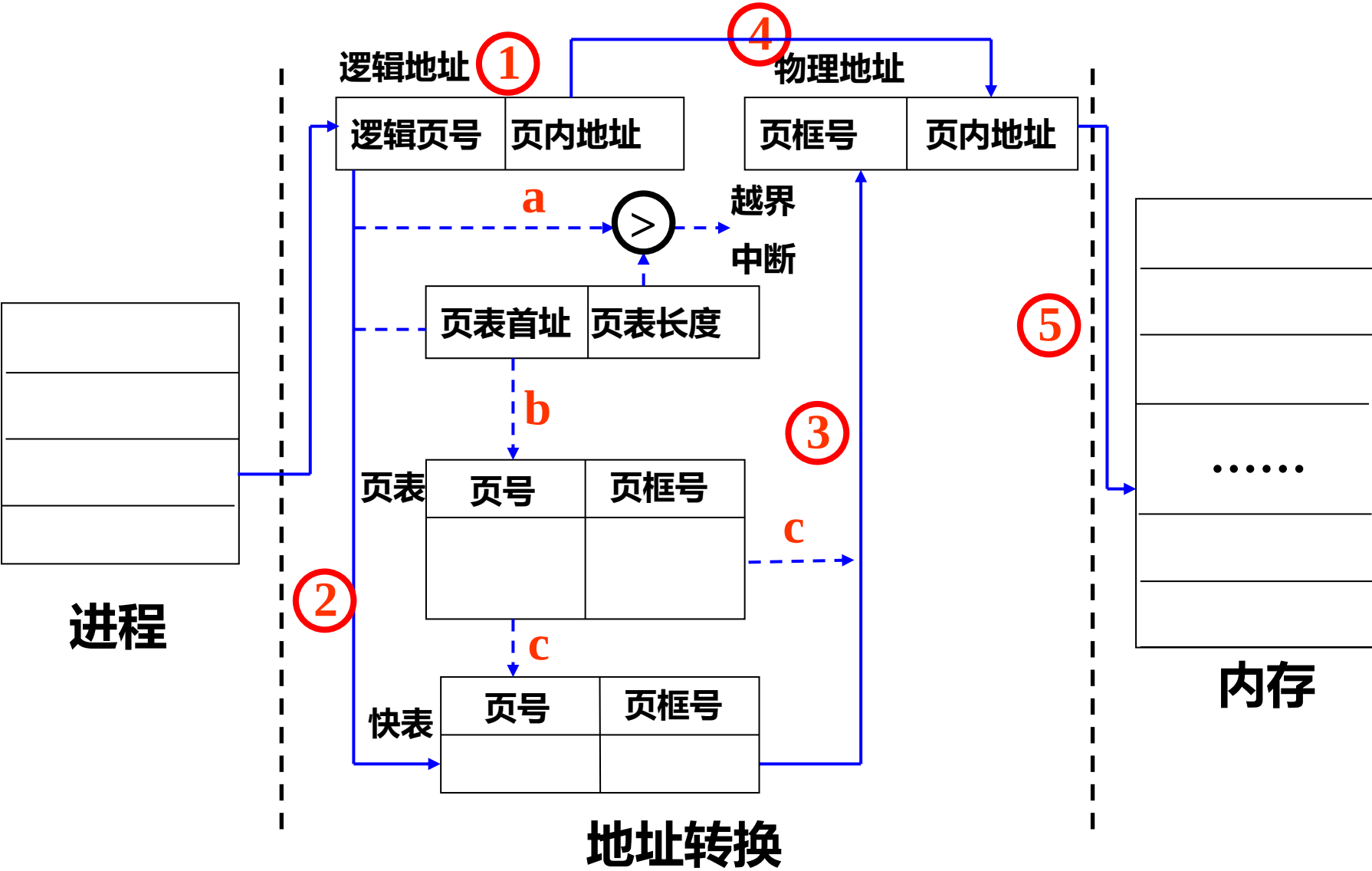
parend

(3) f 与 d 合并得物理地址





地址转换



6.3.2.2 多级页表

■ 提出背景

- 内存空间成倍增长，进程虚拟空间成倍增加
 - 单级页表需要很大连续内存空间
- 例：32位进程地址空间（4G进程空间），页长占12位（4k），页号20位，页表最多可达 2^{20} 个入口！即1M条记录。假设每条记录占4字节，则页表需要占内存 $1M \times 4 \text{ 字节} = 4M$ 。页表占的页框数 $4M / 4k = 1K$ 页框（连续）
 - 问题
 - 即需要找到连续1K个页框存放页表
 - 页表全部装入过度消耗内存（4M）

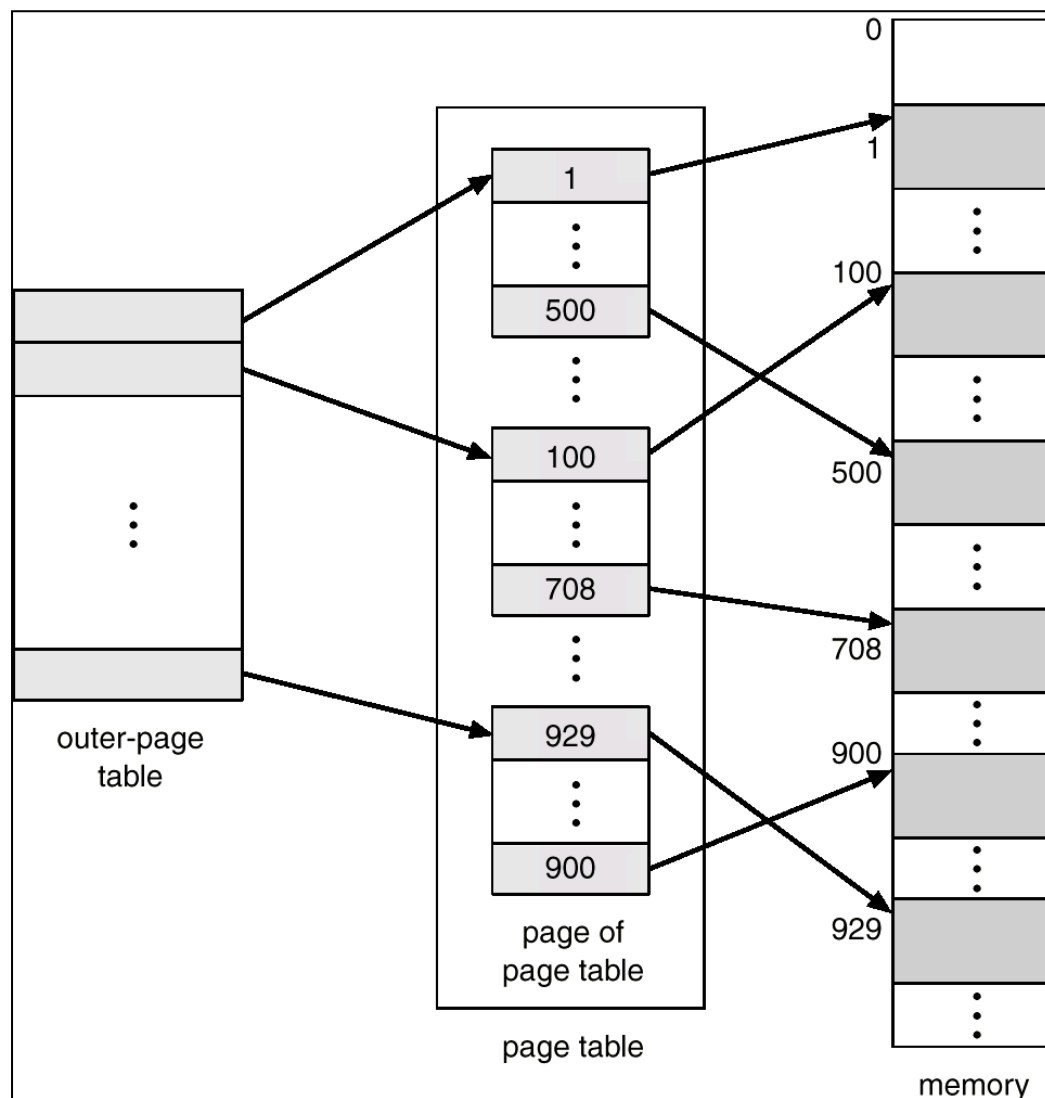
■ 解决策略：二级或多级页表

- 将4M的超大页表存储到离散的1K个页框中
- 仅将页表的部分内容调入内存.

6.3.2.2 多级页表

- 提出背景
 - 多线程设计导致进程虚拟空间不连续（空洞hole）
 - 栈扩展的预留空间（逻辑空间，没有页架相对应）
 - 例如：线程栈开始分配4k，预留1020k
 - 4k有物理页框相对应，1020k没有（溢出时分配）
 - 栈的逻辑地址预留1024k（1020k空洞）
 - 空洞对应的页表项浪费内存空间

Two-Level Page-Table Scheme



- 对页表进行分页，称页表页，长度与页框相同(4K)，可移到外存
- 每个页表页有1024个入口，外页表也有1024个入口，每个入口4字节。总页表项一共是 2^{20} 。页表页之间可以不连续
- 外页表入口指向了页表页的起始地址(物理页框号)

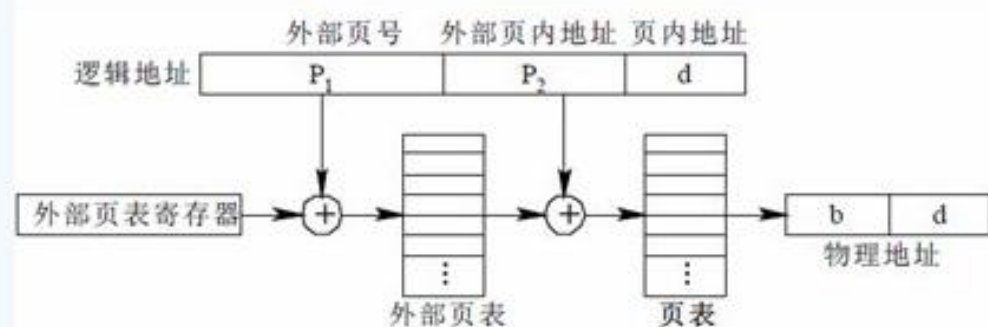
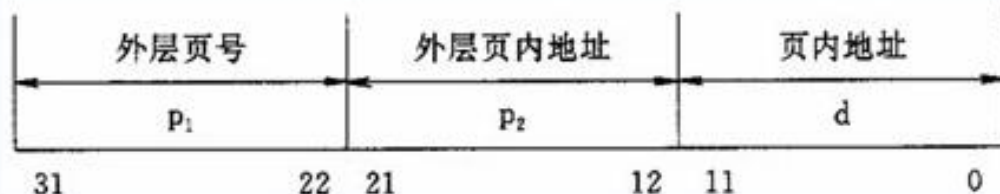
6.3.2.2 多级页表(Cont.)

■ 多级页表

- 多级页表：将大页表进行分页。内存仅存放当前使用的页表，暂时不用的部分可以放在磁盘上，用时再调入。
- 系统为每个进程建立一张页目录表，它的每个表项对应一个页表页，而页表页的每个表项给出了页面和页框的对应关系，此时，页目录表为一级页表（外页表），页表页是二级页表。

1. 两级页表(Two-Level Page Table)

逻辑地址结构可描述如下：



6.3.2.2 多级页表(Cont.)

- 二级页表地址映射特点
 - 访问数据需要三次访问内存（页目录表，页表页，存储单元）
 - 页目录调入内存
 - 页表页按需调入内存
 - 页面、页表页、页目录表的大小都刚好4k（占1个页框）

4级页表有效访问时间

- $EAT = \text{快表命中率} \times (\text{快表访问时间} + \text{内存访问时间}) + (1 - \text{快表命中率}) \times (\text{快表访问时间} + 5 \times \text{内存访问时间}) \text{ ns}$

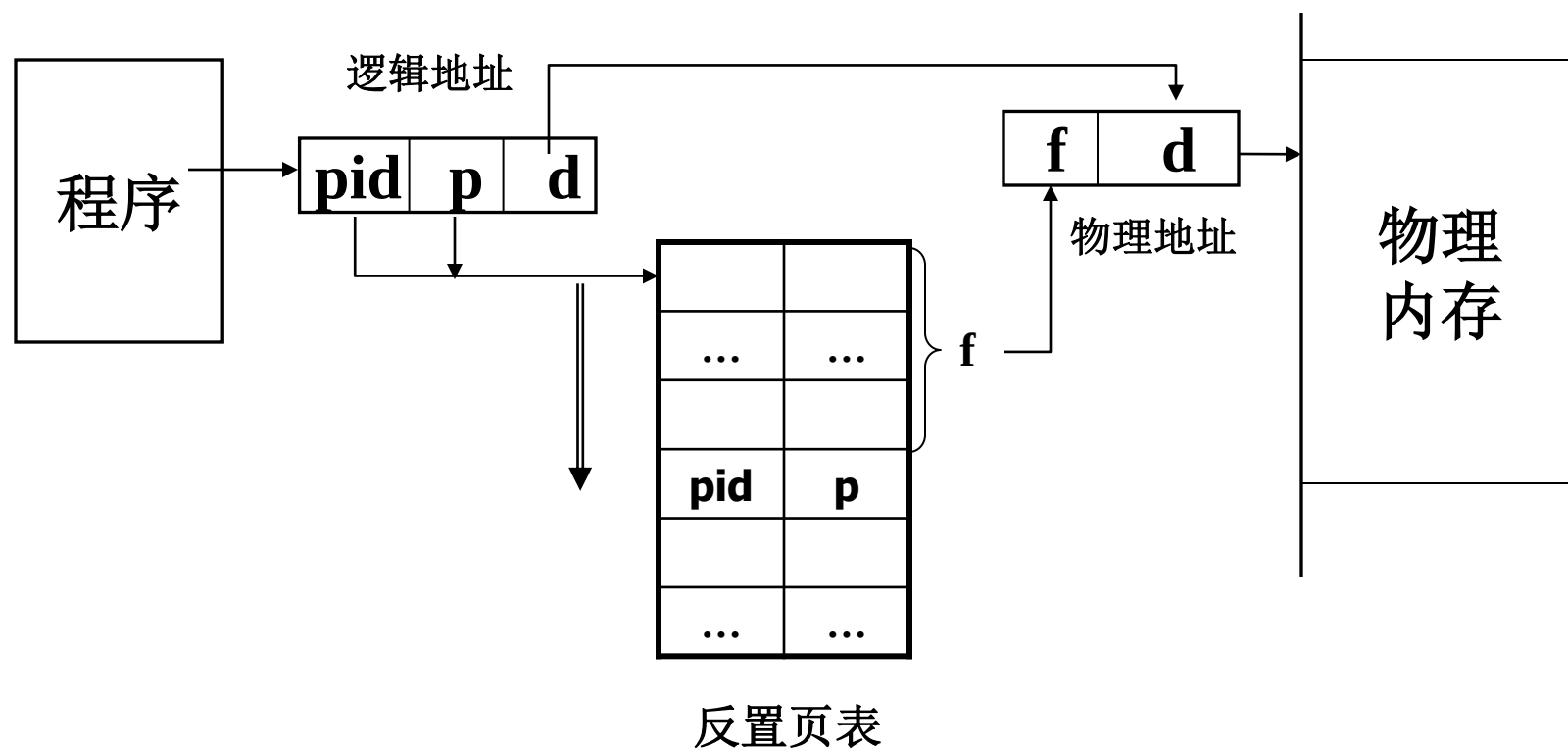
$$98\% \times (20 + 100) + 2\% \times (20 + 500) \text{ ns}$$

$$= 128 \text{ ns}$$

6.3.2.3 反置页表(inverted page table)

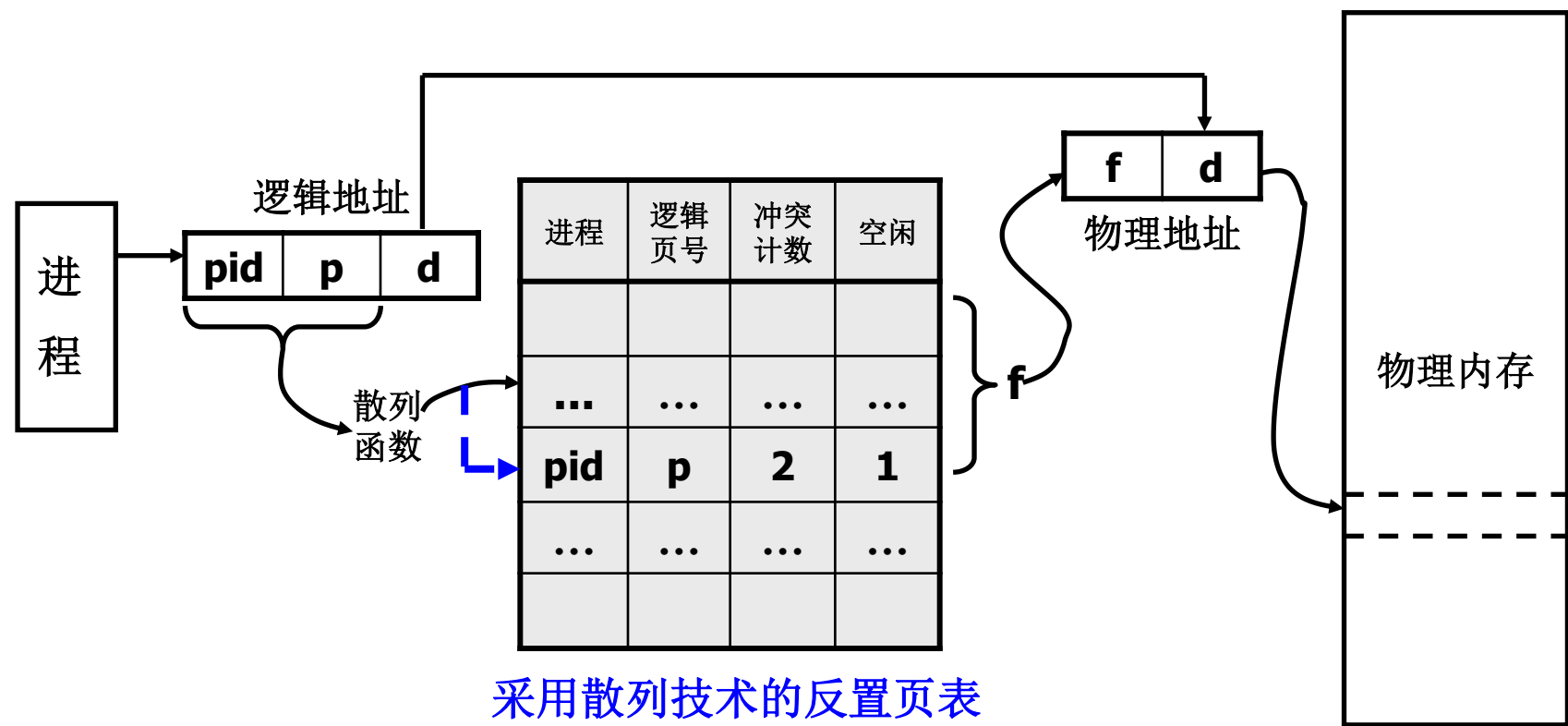
- 传统页表面向进程空间
 - 每个进程逻辑页面有一表项
 - 当进程空间很大时，页表很大
- 反置页表面向内存空间
 - 对每个内存页框设置一个表项，表项的序号为物理页框号 f ，表项的内容为进程标识 pid 与逻辑页号 p 的有序对
 - 反置页表大小固定
 - 整个系统一个反置页表，为所有进程所共用

反置页表--工作原理



6.3.2 页式存储管理(Cont.)

采用杂凑技术的反置页表地址映射



采用散列技术的反置页表
(hash表和反置页表合并)

速度问题

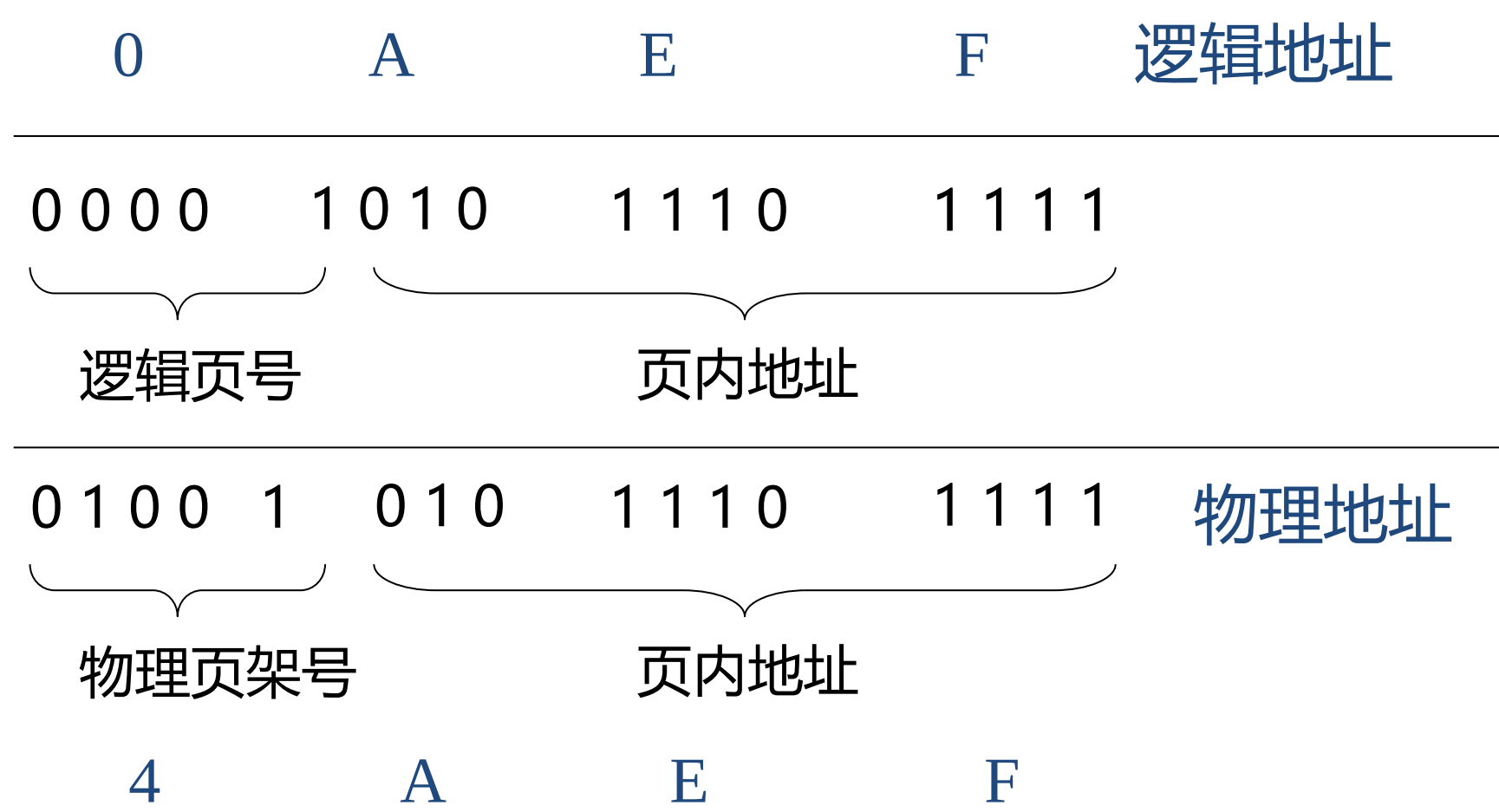
- 反置页表查找
 - 由表头起始，平均为表长度的一半
 - 对于不存在的页需要查找到尾，速度慢
- 解决方案-杂凑(hash)技术
 - 在反置页表前增加一级杂凑表(冲突计数和空闲标志)
 - 在进行地址映射时，由 $\text{hash}(\text{pid}, p)$ 计算得到反置页表入口地址，从入口地址开始向下探查找到对应的表项，位移 f 为对应的页框号
 - 查找杂凑表与反置页表至少需要两次访问内存
 - 用快表缓冲，提高查找速度

页式存储管理实例

❖ 例：系统采用页式存储管理，有一进程大小是8k，页大小2k，依次装入内存的第7，9，2，5块，试将逻辑地址0AEF转换为物理地址

页表	逻辑页号	页框号
	0	7
	1	9
	2	2
	3	5

页式存储管理实例

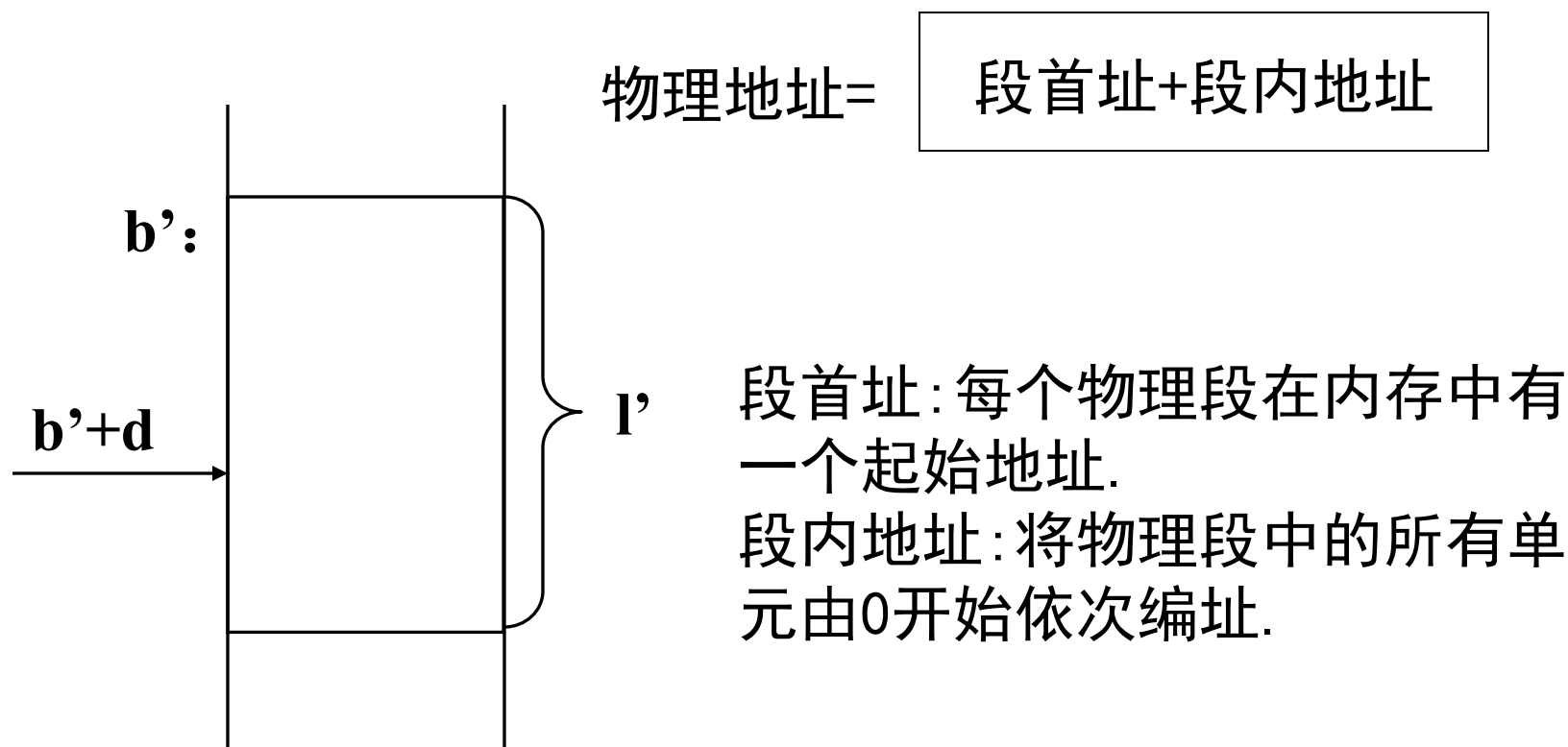


6.3.2 页式存储管理(Cont.)

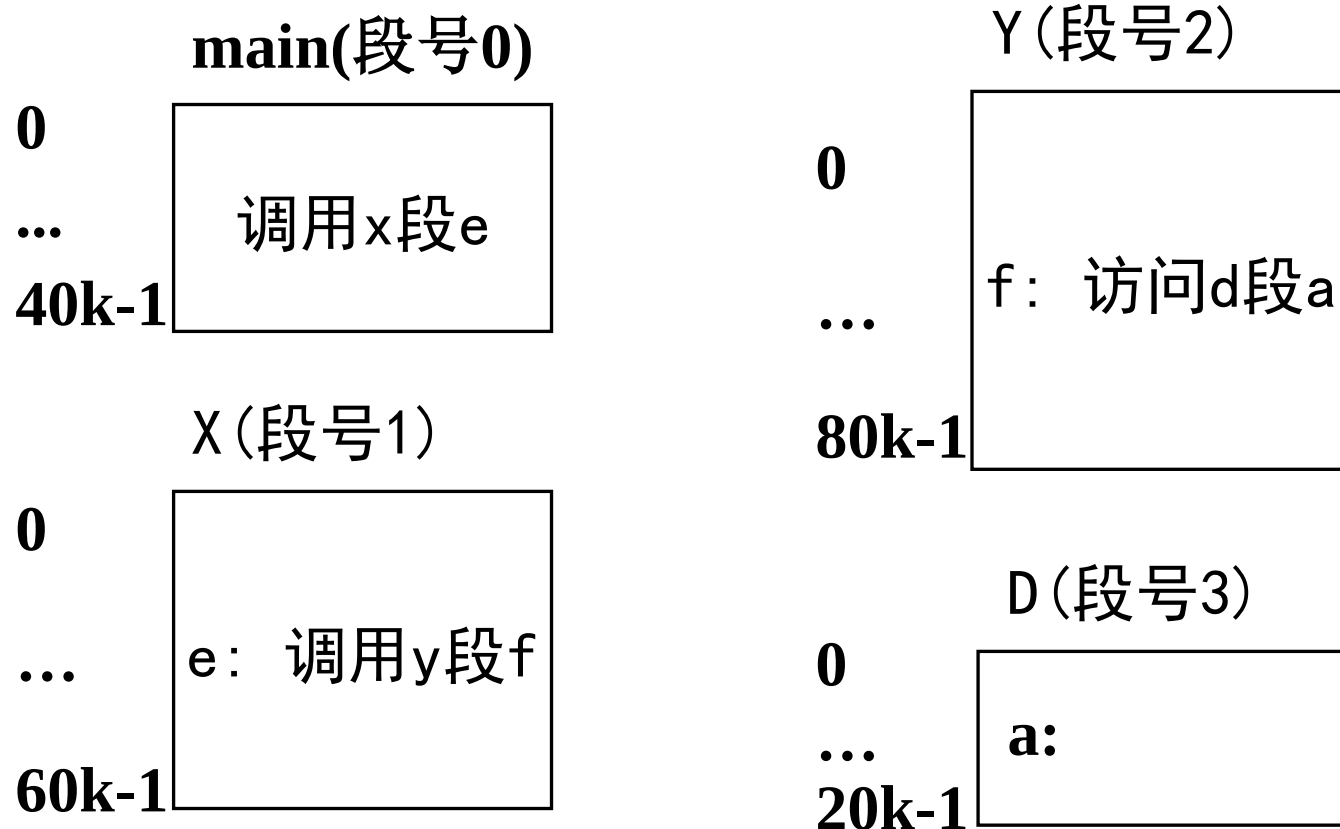
- 优点
 - 消除了外部碎片
- 缺点
 - 一个进程的长度通常并不恰好为页面长度的整数倍，最后一页中会有部分空间未被用，产生页内零头
 - 一个进程的页内零头的长度为半页。

6.3.3 段式存储管理(segmentation)

1. 内存空间划分：动态异长，每区一段。



2. 进程空间划分：若干段，每段一个程序单位。

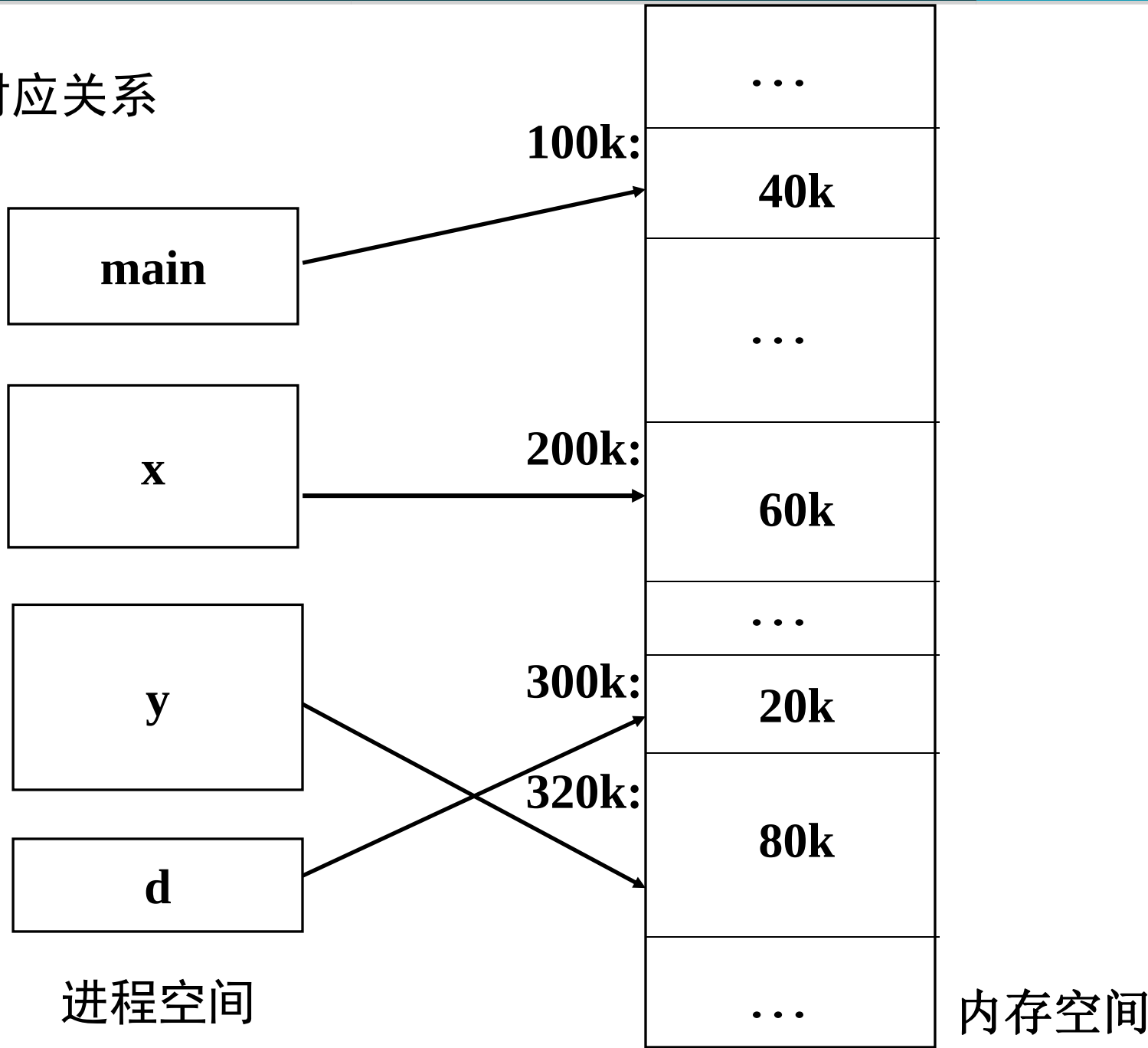


逻辑地址=

段号	段内地址
----	------

 (二维地址)

3. 对应关系



4. 所需表目

(1) 段表：每进程一个，记录段号和段首址的对应关系。

段号	段首址	段长度
0:	100k	40k
1:	200k	60k
2:	320k	80k
3:	300k	20k

(2) 空闲表：系统一个，用于记录并管理系统中的空闲区域。

管理方法：FF，NF，BF，WF

array of (addr, size)

5. 所需寄存器

(1) 段表首址寄存器：用于保存正在运行进程的段表首址

b

 系统一个

(2) 段表长度寄存器：用于保存正在运行进程的段表长度

l

 系统一个

(3) 快表(TLB)：系统一组，用于保存正在运行进程的段表中的部分项目

段号	段首址	段长度
...	...	...
s	b'	l'
...	...	...

6. 地址映射 $\sigma: (s, d) \rightarrow (b' + d) \cup \{\Omega\}$
 逻辑地址 $(s, d) \Rightarrow$ 物理地址 $(b' + d)$

(1) 由程序确定逻辑地址 (s, d) ;

(2) 由段号 s 查快表得段首址 b' 和段长 l'

如查不到:

(a) 由段号 s 与段表长度 l 比较判断是否越界
 不满足: $0 \leq s \leq l-1$, 越界;

(b) 由段号 s 和段表首址 b 查段表, 得段首址 b' 和段长 l'

(c) 由 d 与 l' 比较, 判断是否越界
 不满足: $0 \leq d \leq l' - 1$, 越界;

(d) parbegin

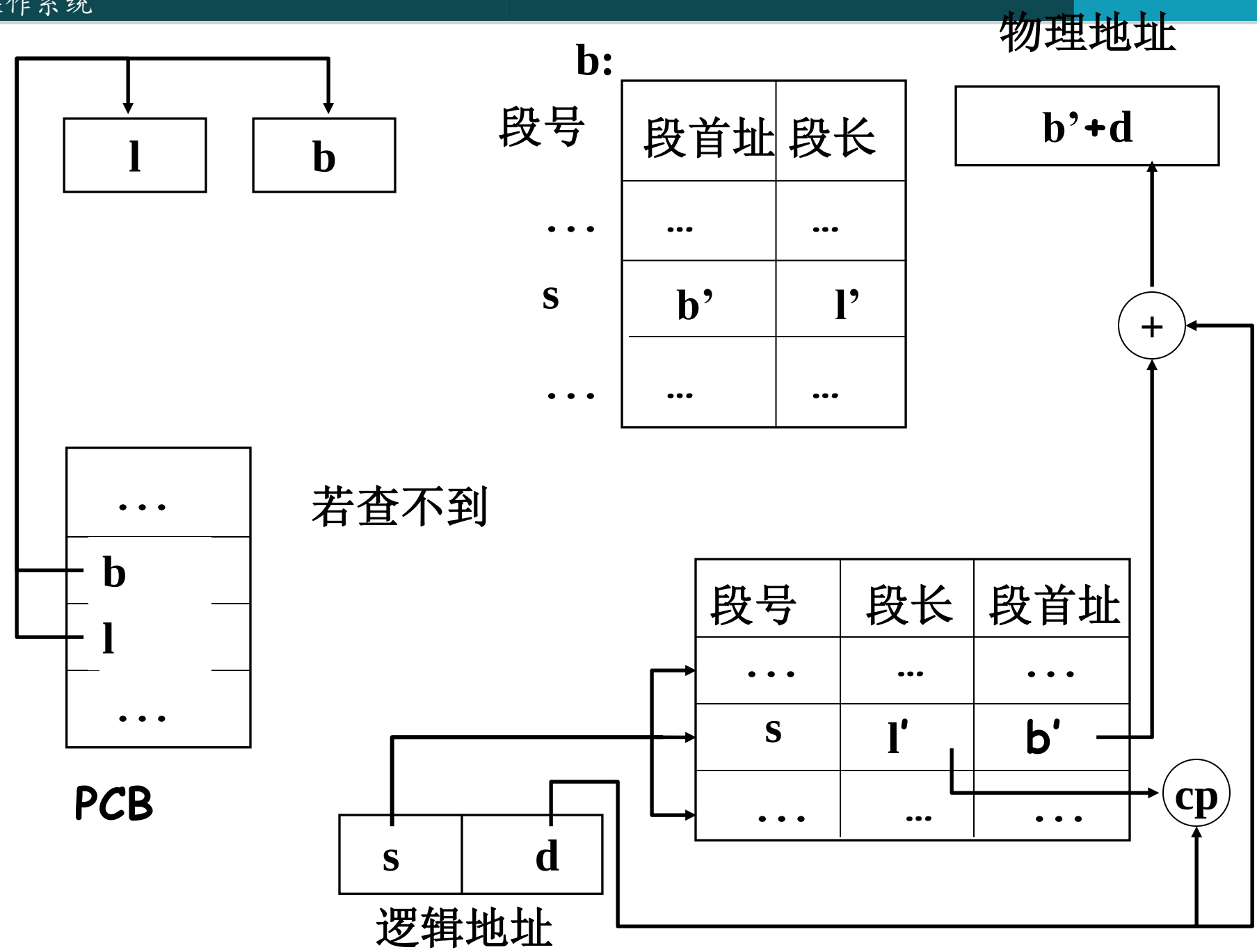
由 $b' + d$ 得物理地址

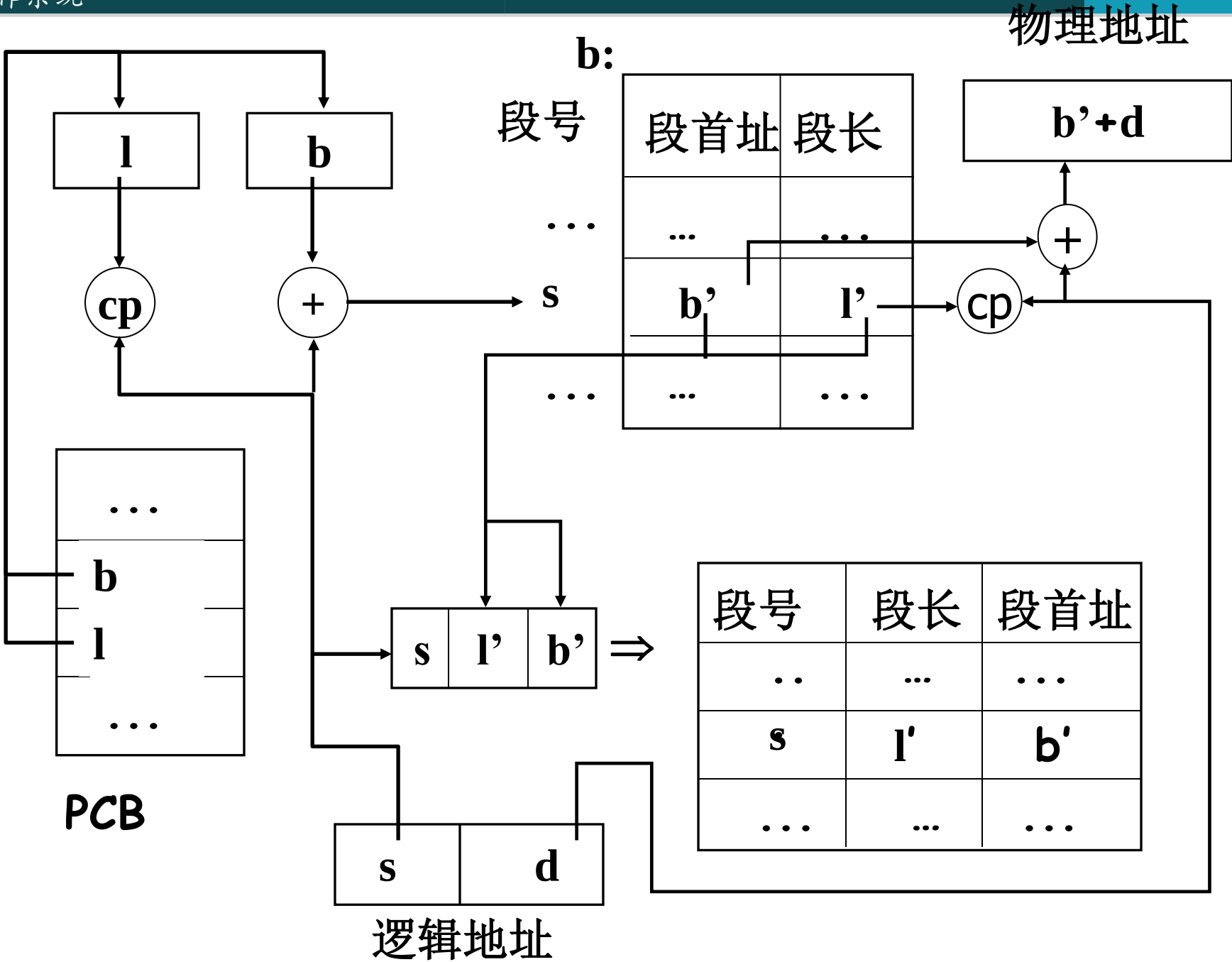
$(s, b', l') \Rightarrow$ 快表, 如快表满淘汰一个;

parend

(3) 由段内地址 d 与段长 l' 比较, 判断是否越界
 不满足: $0 \leq d \leq l' - 1$, 越界;

(4) 由 $b' + d$ 得物理地址。





6.3.3.2 段的共享

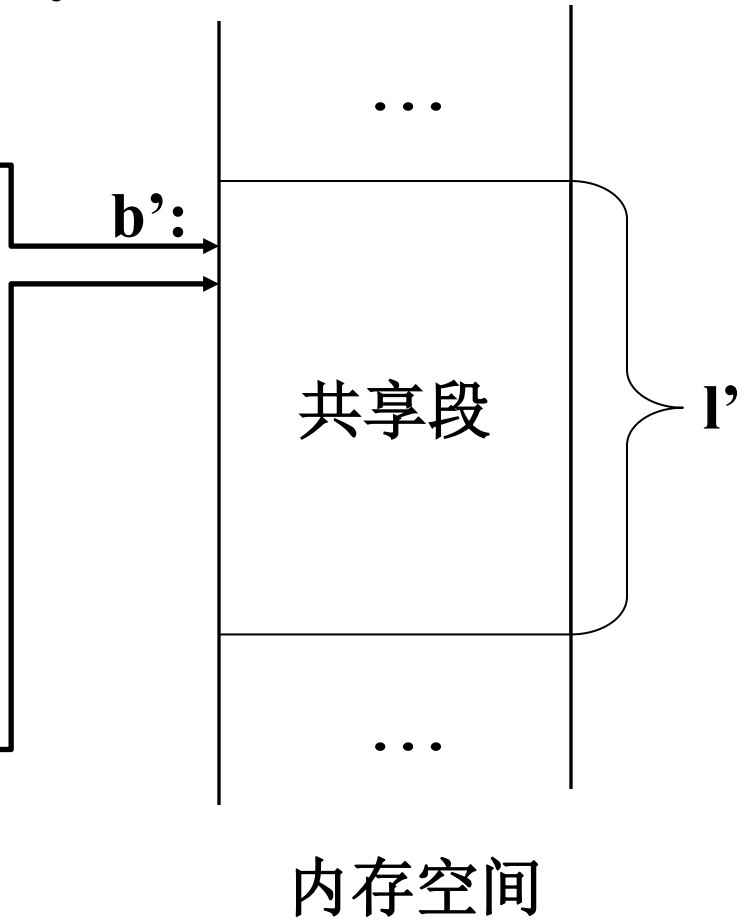
P1段表:

段号	段长	段首址
...	...	...
si	l'	b'
...	...	...

P2段表:

段号	段长	段首址
...	...	...
sj	l'	b'
...	...	...

如何实现?
共享段表



6.3.3.2 段的共享(Cont.)

■ 共享段表

- 进程段表(n) → 共享段表(1) → 共享段(1)
- 当一个共享段被初次使用时，被登记在共享段表中，共享计数为1
- 以后其它进程访问此段时，共享计数加1
- 当一个进程结束对某一共享段的访问时，其共享计数减1
- 当共享计数减到0时，表示没有进程再使用该段，可以释放所占用的存储空间

段名	共享记数	段长	段首址	其它
...	...	...	...	...
vi	3	35k	125k	??
...	...	...	...	...

链接共享段时：段名查共享段表

(1) 找到：已有进程在用

(a) 共享计数加1；

(b) 段表指向共享段表

(2) 未找到

(a) 读入内存

(b) 填共享段表，共享记数=1

(c) 段表指向共享段表

6.3.3.2 段的共享(Cont.)

- 共享段表功能
 - 用来寻找共享段
 - 确保一个共享段只有一组描述信息

P1段表

段号	段长	首址	共享入口
...	...	...	...
si	l'	b'	
...	...	...	...

P2段表

段号	段长	首址	共享入口
...	...	...	...
sj	l'	b'	
...	...	...	...

段名	共享计数	段长	段首址
...	...	...	...
vi	2	35k	125k
...	...	...	...

125K:



例子：UNIX正文段(text段)

```
struct text
{
    int  x_daddr;    /*disk address
    int  x_caddr;    /*core address, if loaded
    int  x_size;     /*size(×64)
    int  *x_iptr;    /*inode pointer
    char x_count;    /*reference count
    char x_ccount;   /*number of loaded reference;
}text[NTEXT];

define NTEXT 40
```

```
struct proc  
{
```

```
.....
```

```
int  *p_textp;  /*pointer to text structure;
```

```
}
```

```
struct user
```

```
{
```

```
.....
```

```
int  u_tsize;
```

```
.....
```

```
}
```

6.3.3.2 段的保护

- 进程对于共享段的访问往往需要加上某种限制
 - 对于保存共享代码的段，任何进程都不能修改它
 - 对于具有保密要求的段，某些进程不能读取它
 - 对于属于系统数据的段，某些进程不能修改等

6.3.3.2 段的保护

(1) 段表的改进：段号

	段长	段首址	共享段 表入口	访问权限		
				R	W	E
...	...	...	...	..	..	..
s	l'	b'	e	1	0	1
...	...	...	...	..	..	..

(2) 快表的改进：

段号	段长	段首址	访问权限		
			R	W	E
...	...	...	..	..	..
s	l'	b'	1	0	1
...	...	...	..	..	..

6.3.4 段页式存储管理

- 段式优于页式
 - 便于共享和保护
- 页式优于段式
 - 消除“碎片”问题
- 段页式：结合二者优点
 - 每个进程包含若干段
 - 每个段包含若干页

6.3.4 段页式存储管理

- 基本原理

1. 内存空间划分：（同页式）

静态等长， 2^i ，称为一页框。

物理地址 = (页框号, 页内地址) = (f, d)

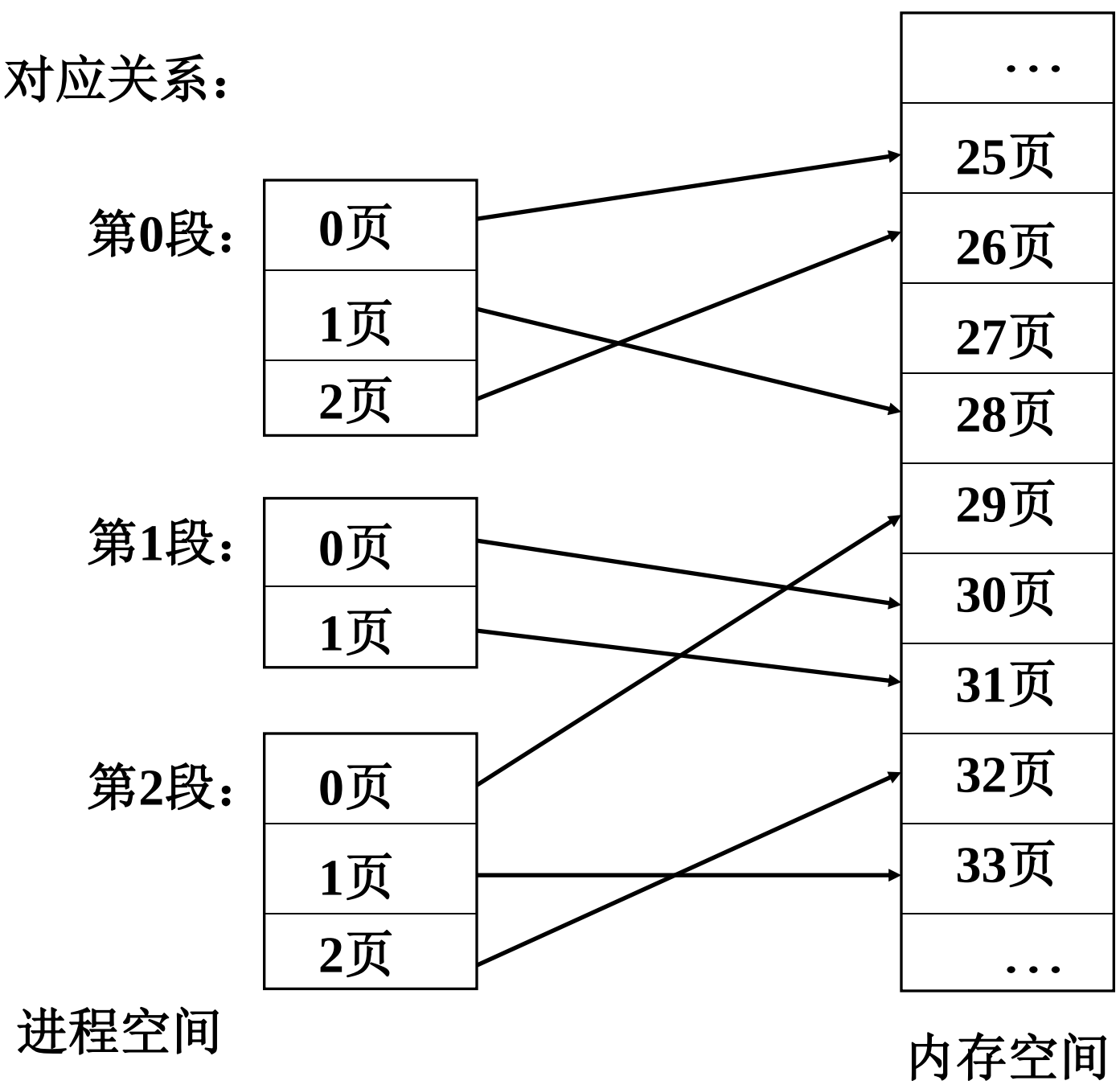
2. 进程空间划分：

一个进程 $\Leftrightarrow$ 若干个段

一个段 $\Leftrightarrow$ 若干个页

逻辑地址 = (段号, 逻辑页号, 页内地址) = (s, p, d)

3. 对应关系:



4. 所需表目

(1) 段表：每个进程一个段号

0
...
s
...
l-1

页表首址	页表长度
...	...
b'	l'
...	...

(2) 页表：每个段一个 逻辑页号

0
...
p
...
l'-1

页框号
...
f
...

(3) 总页表：系统一个，用于记录并管理内存页面

5. 所需寄存器

(1) 段表基址寄存器：保存正运行程序段表首址；

<i>b</i>

(2) 段表限长寄存器：保存正运行程序段表长度。

<i>l</i>

(3) 快表：一组联想寄存器（快段表+快页表）（TLB），保存正在运行进程的段表和页表中的部分表目

段号	逻辑页号	页框号
...	...	...
s	p	f
...	...	...

6. 地址映射 $\sigma: (s, p, d) \rightarrow (f, d) \cup \{\Omega\}$

逻辑地址 $(s, p, d) \Rightarrow$ 物理地址 (f, d)

- (1) 由程序确定逻辑地址 (s, p, d) ;
- (2) 由段号和逻辑页号 (s, p) 查快表得页框号 f ;

如找不到:

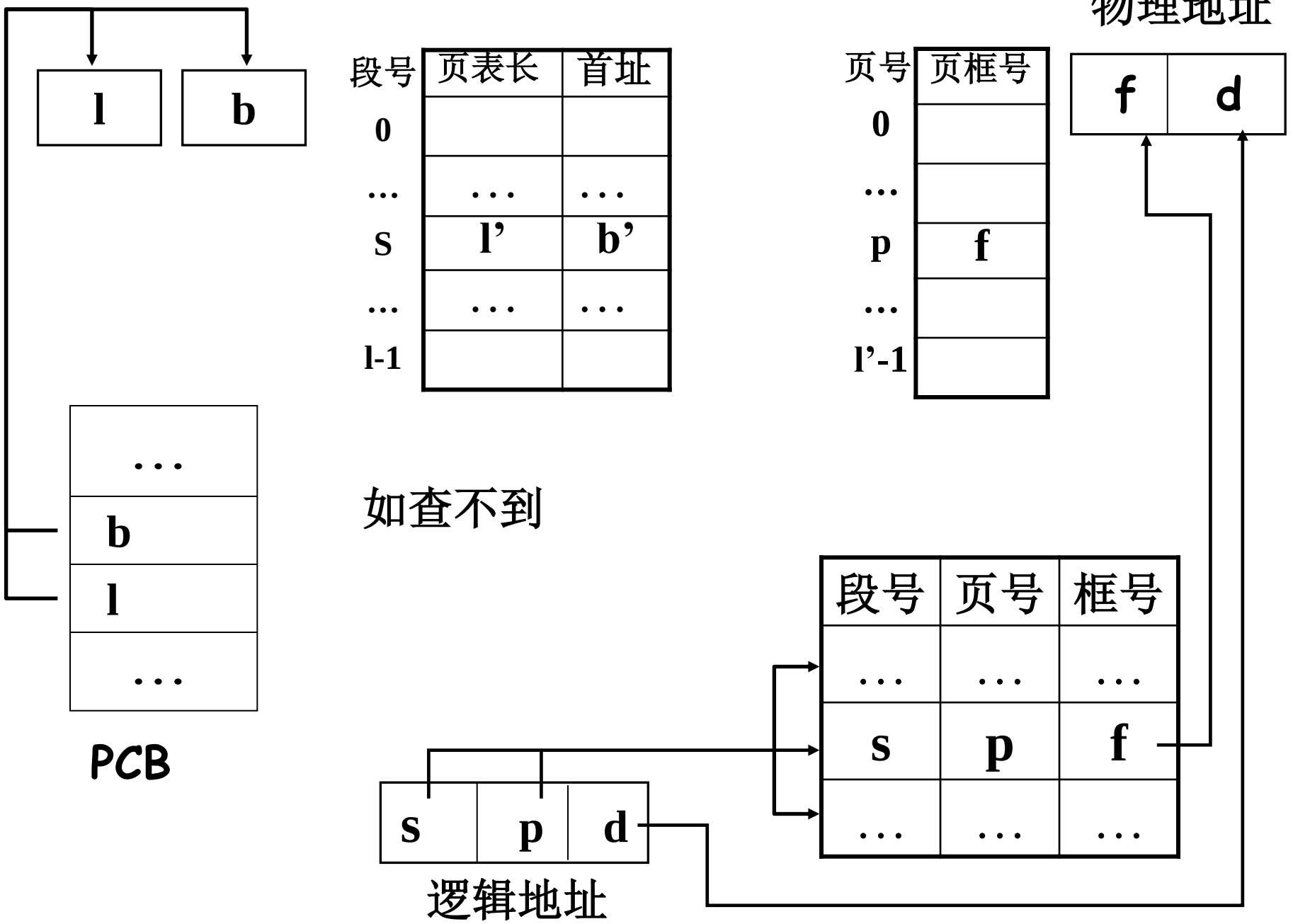
- (a) 由段号 s 与段表长度 l 比较判断是否越界:
不满足: $0 \leq s \leq l-1$, 越界
- (b) 由段号 s 和段表首址 b 查段表得页表 (b', l')
- (c) 由逻辑页号 p 与页表长度 l' 比较判断是否越界:
不满足: $0 \leq p \leq l'-1$, 越界
- (d) 由页表首址 b' 与逻辑页号 p 查页表得物理页框号 f
- (e) parbegin

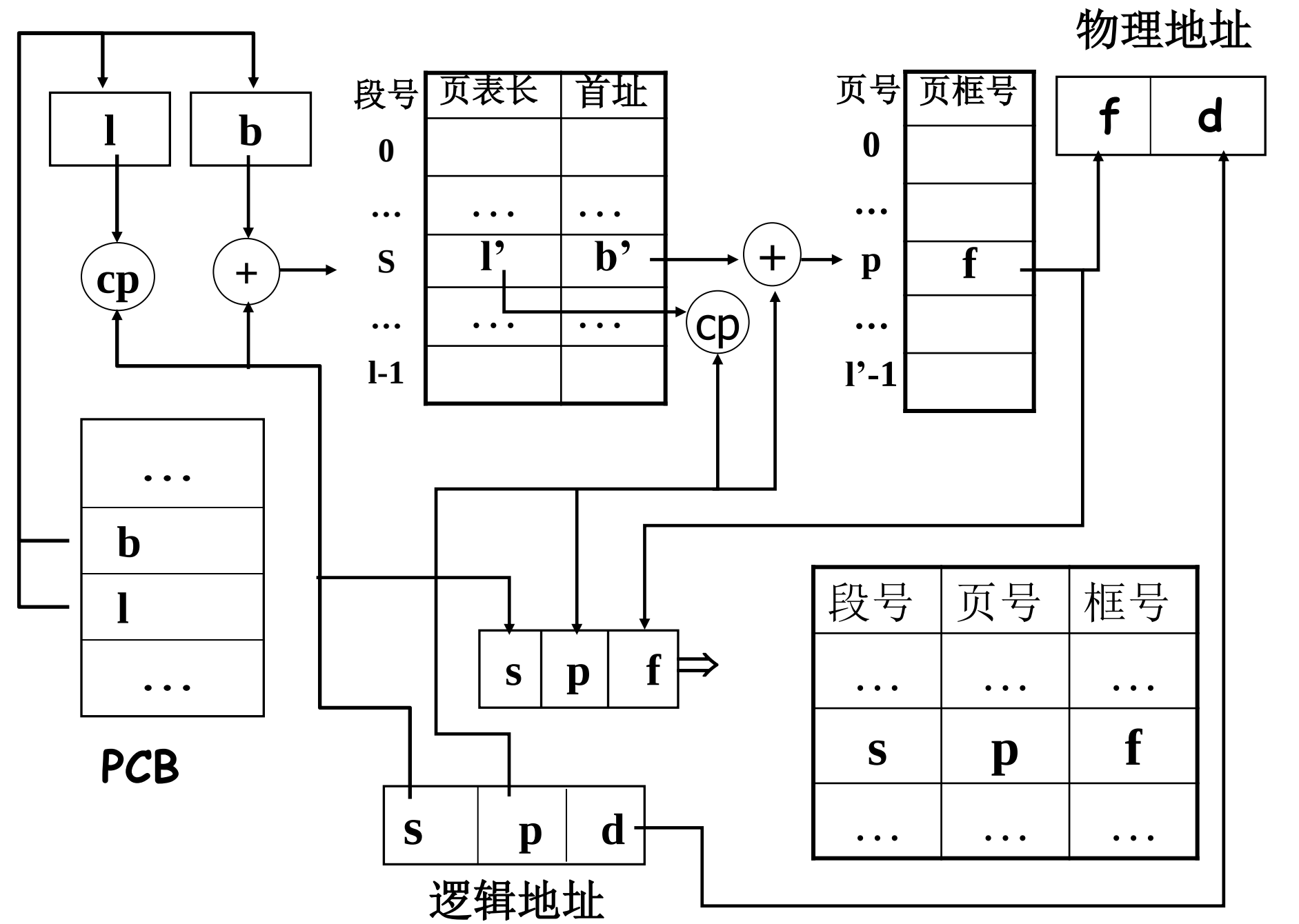
页框号 f 与页内地址 d 合并得物理地址

$(s, p, f) \Rightarrow$ 快表, 若快表已满, 淘汰一个

parend

- (3) 由页框号 f 与页内地址 d 合并得物理地址 (f, d)





段的共享 (Cont.)

- 例：文本编辑器占多少内存
 - 文本编辑器：150KB代码段和50KB数据段
 - 有10个进程并发执行该文本编辑器
 - 占用内存=10* (150+50) KB=2M
 - 如果采用代码段共享，代码段在内存只有一份真实存储
 - 占用内存=150+10*50=650KB。

6.4 系统举例- Linux伙伴堆存储分配算法

- Linux 采用DMA方式进行输入输出操作，DMA不带有地址变换机构，即是在没有地址映射的条件下进行的，因此进程在内存中必须占有连续的页面
- 针对Linux系统，采用伙伴堆存储分配方法。即对内存空闲页面管理时，将连续的页面放在一组，这就是伙伴堆的思想
- 伙伴堆算法用于管理内存中的空闲页框，它是针对内存**碎片问题**而提出的一种稳定高效的分配策略

1. 伙伴堆存储分配算法

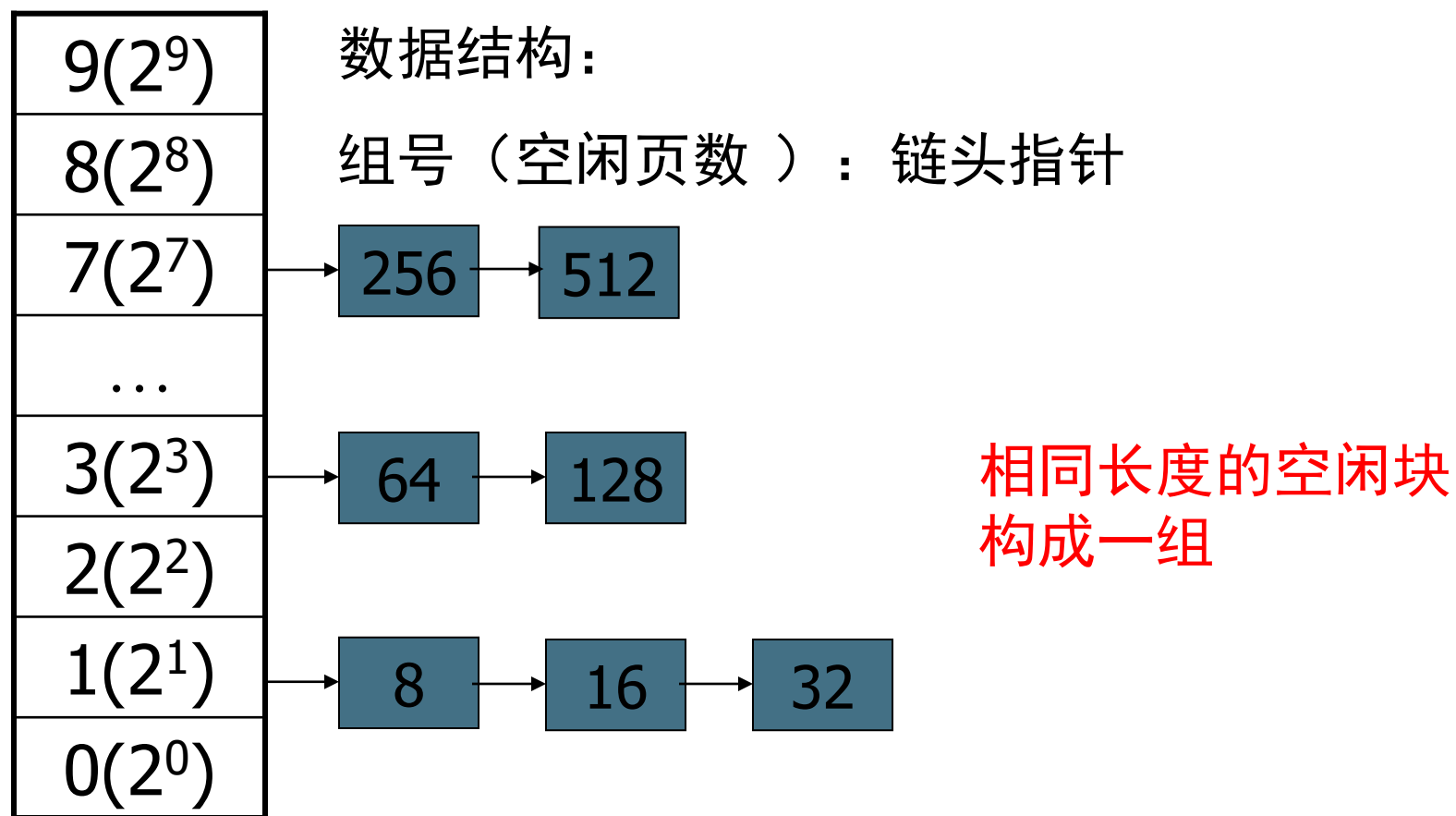
(1) Physical memory management

- 页框：将内存静态划分为若干等长的区域, 每个区域为一个页框, 例如: 4KB;
- 块组：Linux将所有空闲页面分为10个块组, 块组编号为 i ($i=0, 1\cdots 9$),
块组 i 中记载长度为 2^i 个页面的连续区域, 即第0组中块的大小为 2^0 (1页), 第1组中块的大小均为 2^1 (2页), 第9组中块的大小均为 2^9 (512页), 同组中的所有块以链表形式存储。
- 空闲区表: `free_area[i]`表示页框数为 2^i 的块组, 其结构为:

空闲块组指针	块组位图指针
--------	--------

每个空闲块组指针指向该块组内的空闲块组链 (每个块组大小为 2^i 页面)
 每个块组位图指针指向该块组内页面的使用情况
- 分配: Buddy heap algorithm
 以 2^i 个页框为分配单位 ($2^{i-1} < fn \leq 2^i$),
 fn 为要申请的页框数;

Buddy Heap Implementation



■ 块组位图：

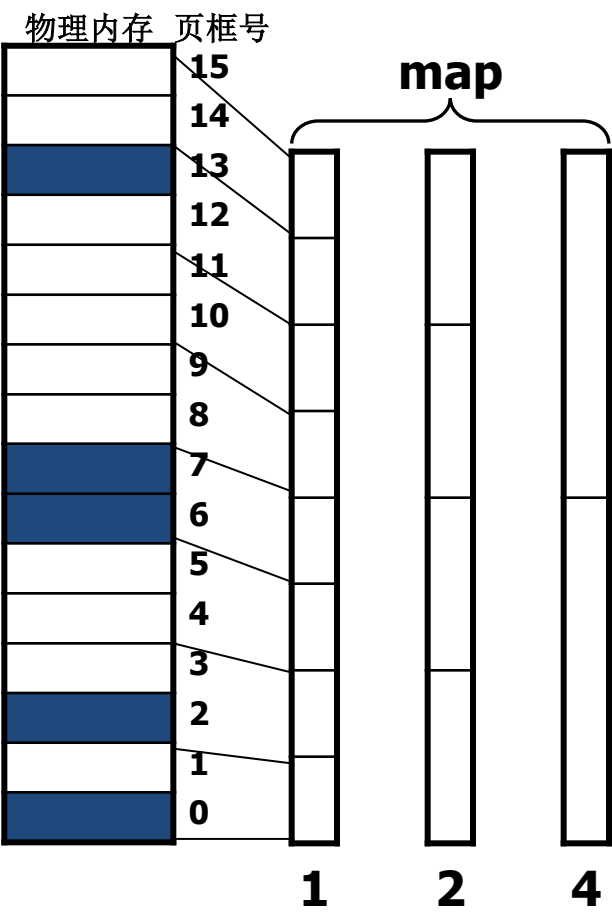
- 对于块组 i ，将内存中的所有页面（包括占用和空闲），按前后顺序两两结合成一对伙伴（Buddy）。即按前后顺序以 2^i 个页面作为一个块组，与其相邻的 2^i 个页面作为另一个块组，那么这两个块组合为一对Buddy。
如：对于块组1，那么0、1页框和2、3页框是一对Buddy；

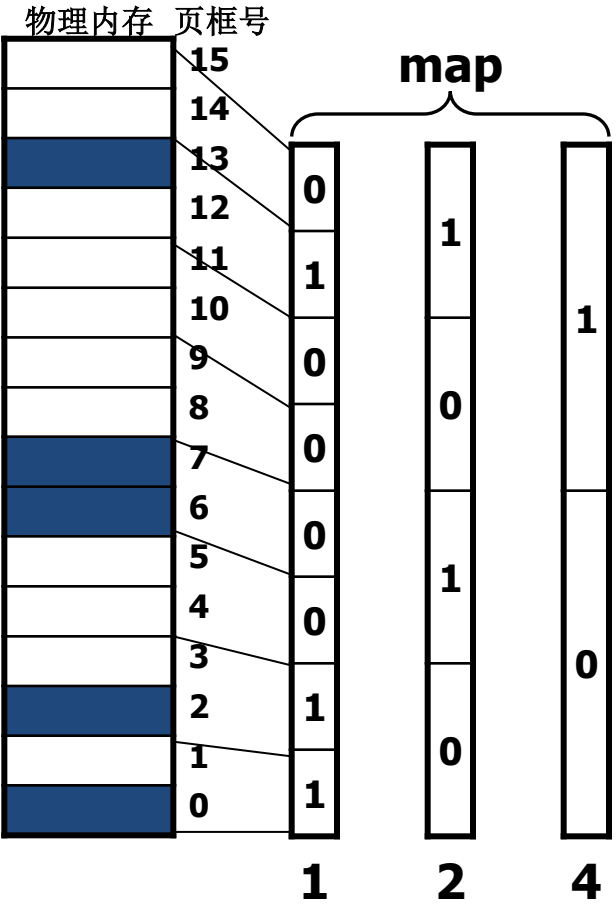
对于块组2，那么0, 1, 2, 3页框与4, 5, 6, 7页框是一对Buddy。

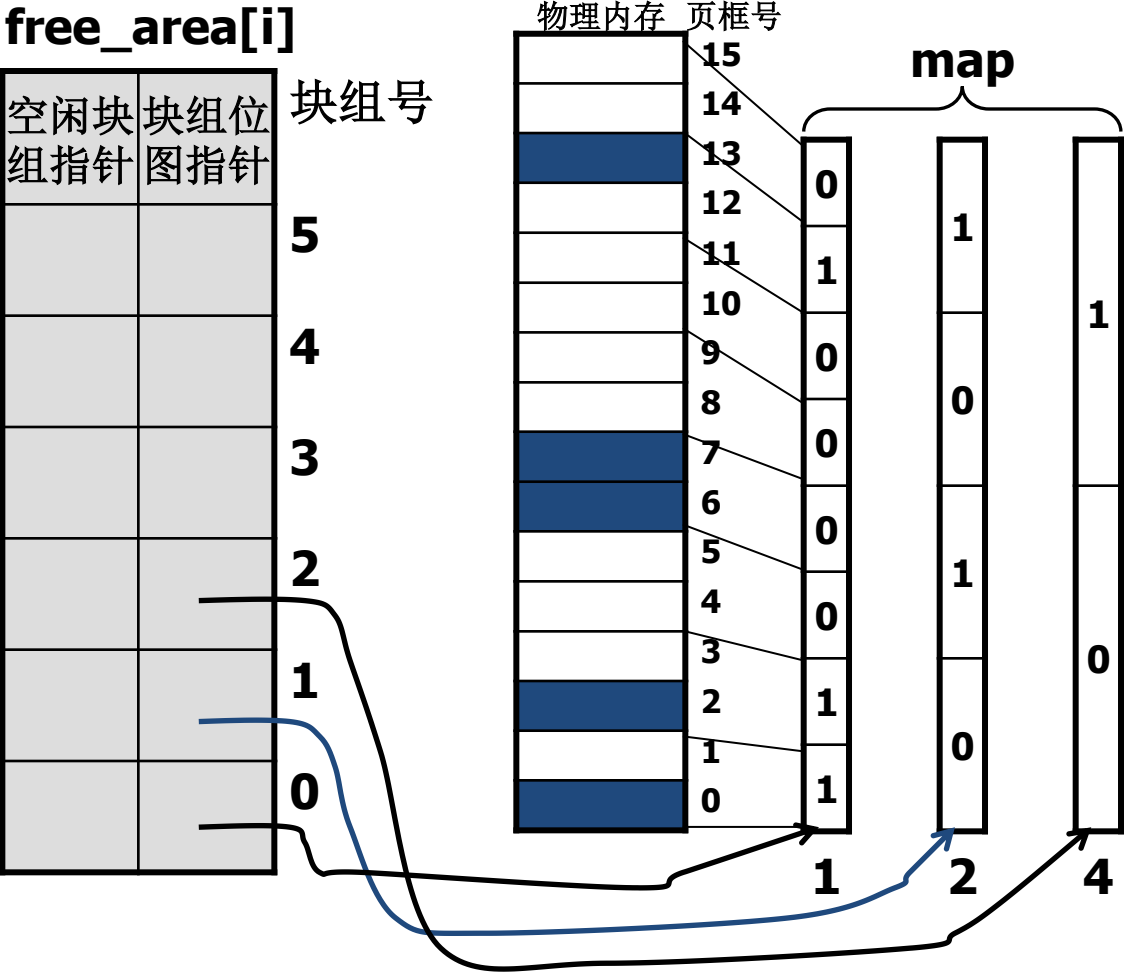
- 块组位图内容表示对应的一对块组伙伴的使用情况；
- 对于一对块组伙伴：
 - ① 若一个空闲，另一个全部或部分占用，则位图相应位置1；
 - ② 当两个都空闲，或都被全部或部分占用，则位图相应位置0。

■ 伙伴条件：

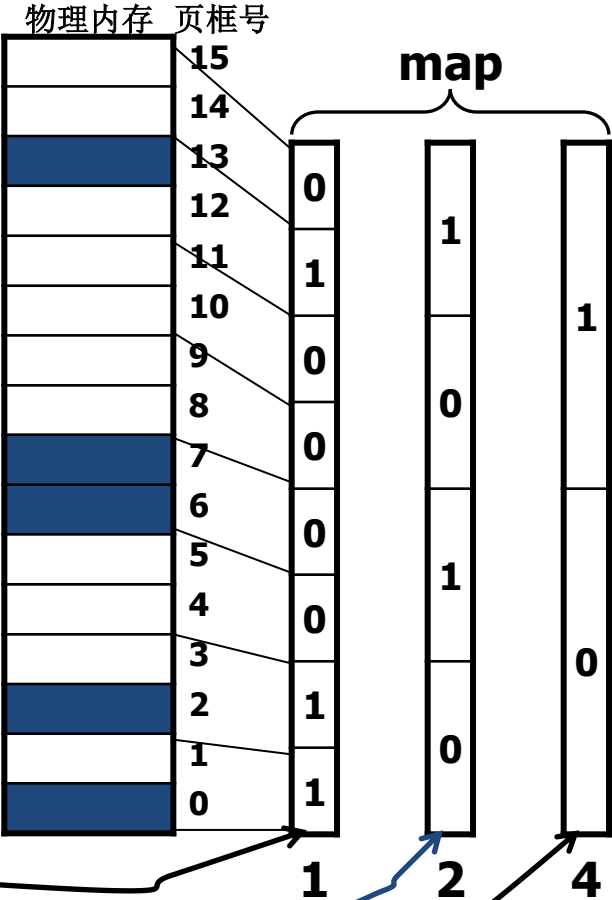
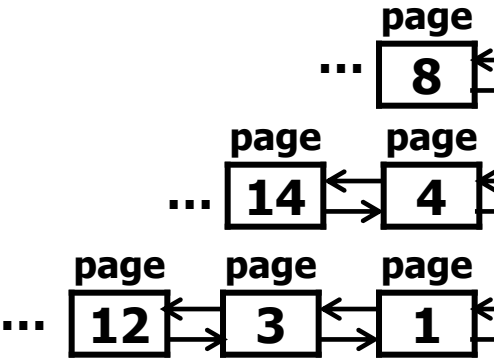
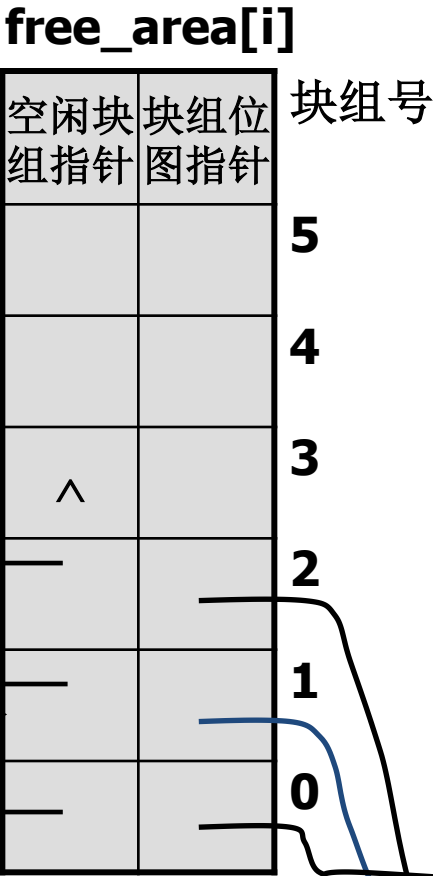
- ① 两个块大小相同，即具有相同的页框数 b ；
- ② 两个块的物理地址相连；
- ③ 如果以0作为页框的初始编号，那么位于后面块组的最后页框编号+1必须是 $2b$ 的整数倍。







■ 空闲区链表
组织结构:
假设6个块组



(2) Buddy heap algorithm

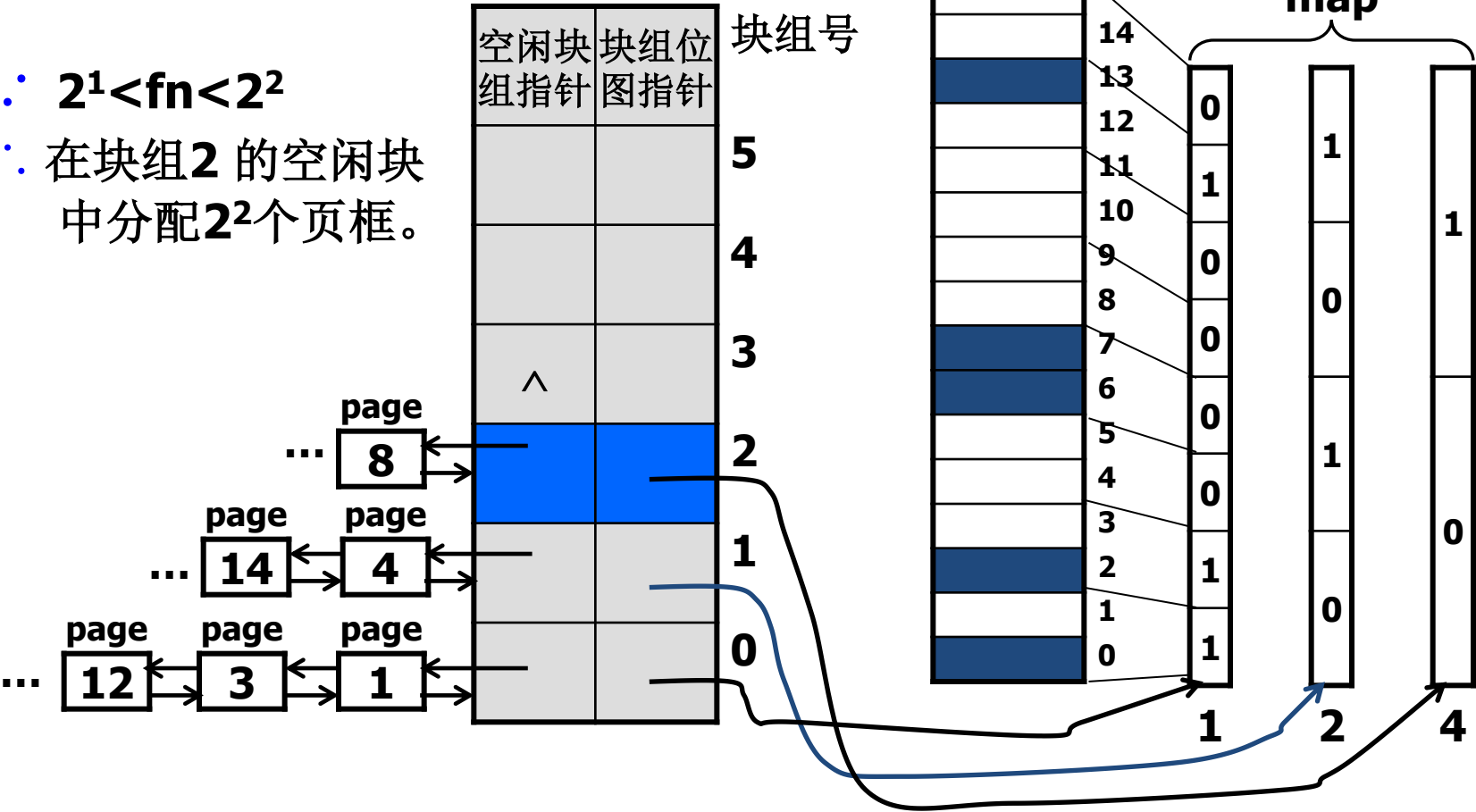
- 分配: 申请 fn 个页框

- ① 计算($2^{i-1} < fn \leq 2^i$), 找到相应的块组 i ;
- ② 在块组 i 的第一个空闲块分配 2^i 个页框;
- ③ 调整块组 i 的空闲块链表;
- ④ 如果块组 i 没有空闲块, 则向块组编号大的块组方向查找空闲块
- ⑤ 若 $2^j > 2^i$, 则把 $2^j - 2^i$ 个空闲页框加入到相应块组空闲链中;
(若 $2^j - 2^i$ 不是2的整数次幂, 则将其拆分成不同的整数次幂。)
- ⑥ 修改位图。

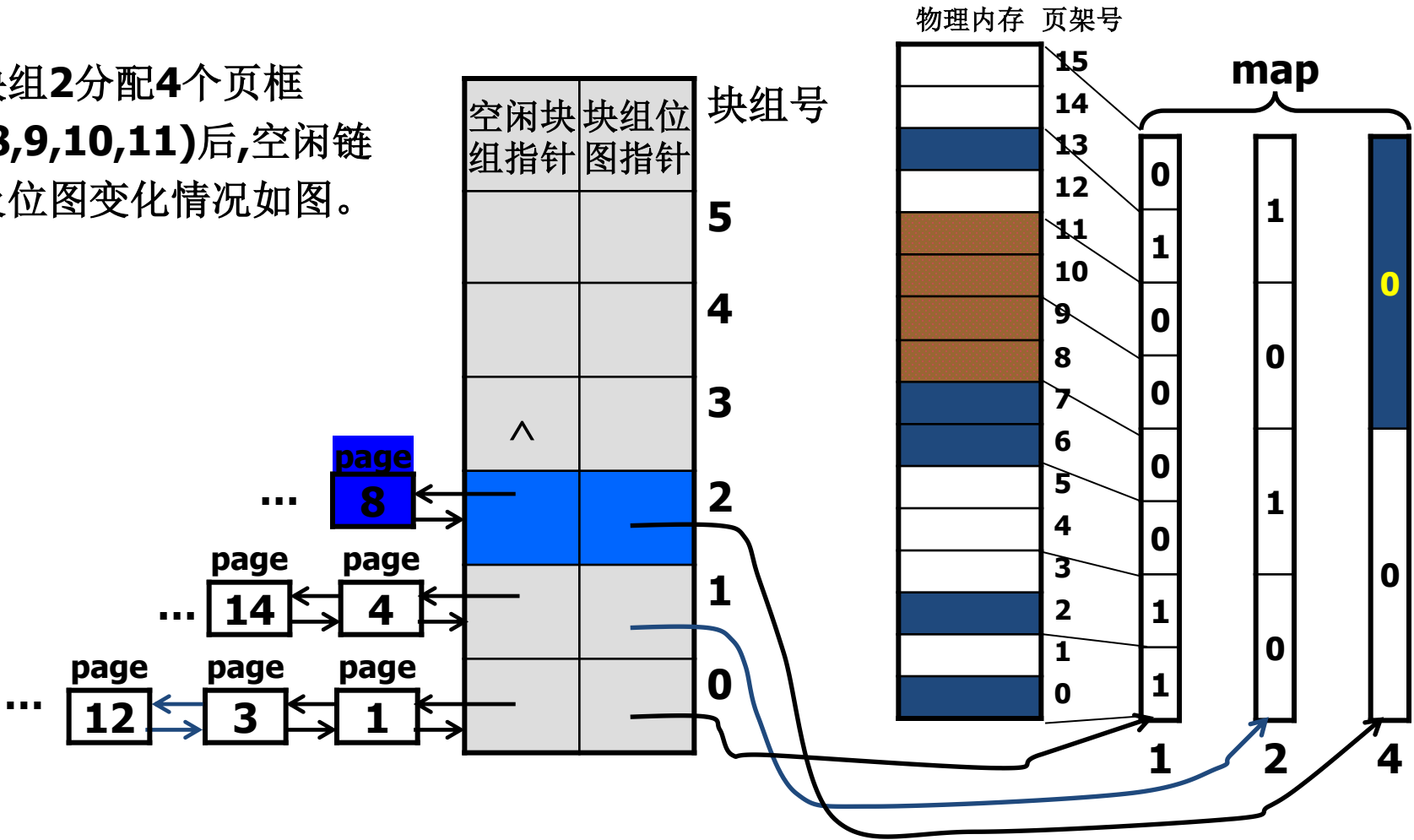
例如: 对于长度为128页的请求, 应该在第7组中取一块分配。如果第7组已空, 取第8组中的一块, 分配其中的128页, 并将剩余的128页加入第7组中。若第8组也空, 取第9组中的一块, 进行两次分割, 分配128页, 将剩余的128页和256页分别计入第7组和第8组。

■ 分配例: 申请页框数fn=3

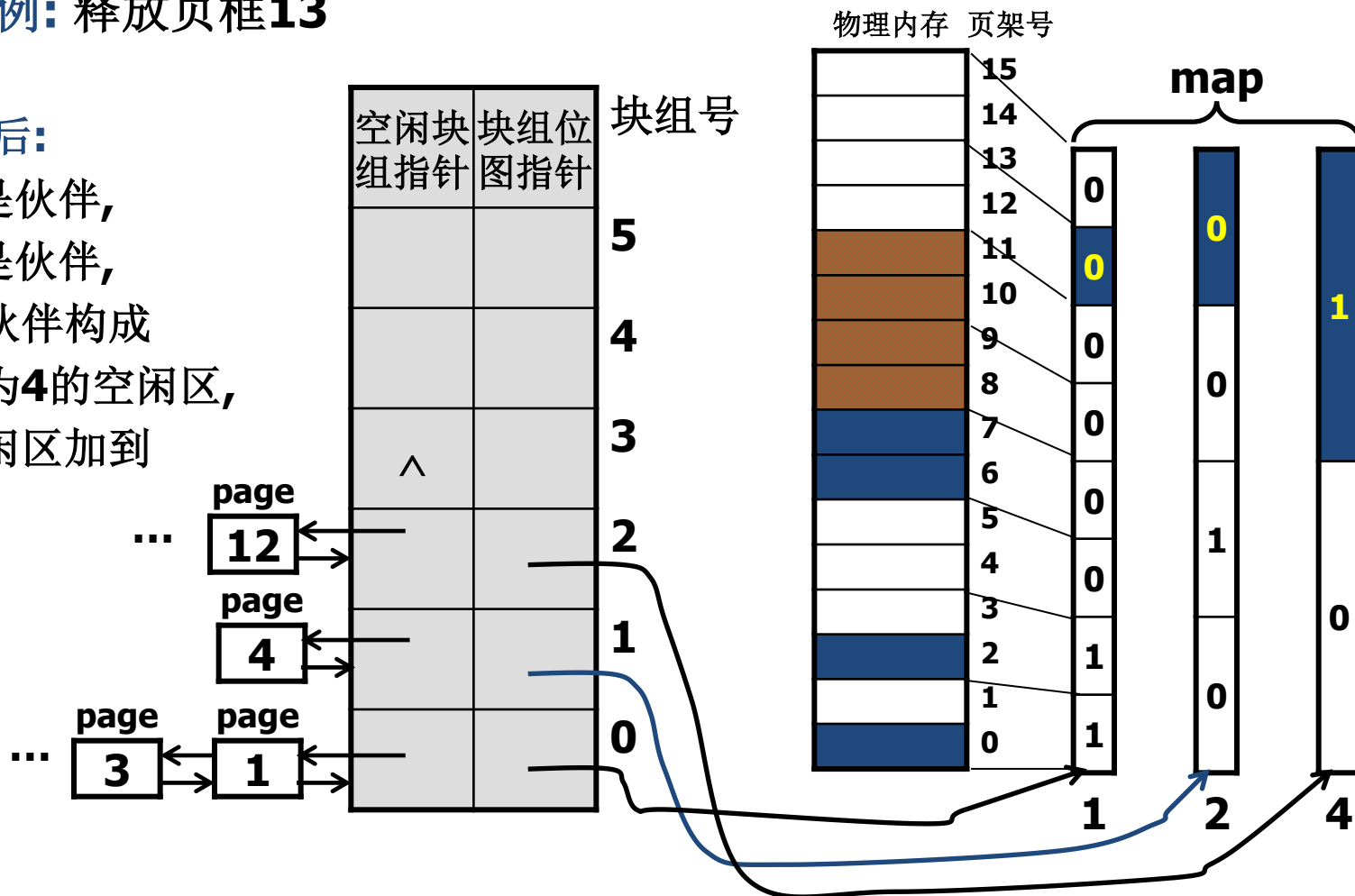
- ∵ $2^1 < fn < 2^2$
- ∴ 在块组2 的空闲块中分配 2^2 个页框。



块组2分配4个页框
(8,9,10,11)后,空闲链
及位图变化情况如图。



12,13是伙伴,
14,15是伙伴,
这两个伙伴构成
页框数为**4**的空闲区,
将该空闲区加到
块组**2**。



■ Buddy heap algorithm:

● 问题: internal fragmentation.

例如: 申请17个页架, 由于 $2^4 < 17 \leq 2^5$,

按Buddy heap algorithm,

要在块组5的空闲区分配32个页框,

造成15个页框的浪费, 即internal fragmentation.

● 解决办法:

① second memory allocator

当实际申请页框数 $fn < 2^i$ 时

将 $2^i - fn$ 按2的整数次幂切分(carves slabs)

由second memory allocator单独管理

② third memory allocator

进程物理空间不要求连续时,

内存分配由third memory allocator完成。

总结

- 存储管理功能
 - 存储分配和去配、存储共享、存储保护、存储扩充、地址映射
- 内存资源管理
 - 静态等长分区的分配
 - 字位映象图、空闲页面表、空闲页面链
 - 动态异长分区的分配
 - 最先适应 (First Fit)、下次适应法 (Next Fit)、最佳适应 (Best Fit)、最坏适应 (Worst Fit)
- 存储管理方式
 - 界地址管理方式(一维地址)、页式管理方式(一维地址)、段式管理方式(二维地址)、段页式管理方式(二维地址)

作业4

1 设某计算机主存有16个页框，内存分配和释放采用伙伴堆算法(Buddy heap algorithm)。主存空闲区表的表项 $\text{free_area}[i]$ 的结构为：

空闲块组 i 的位图指针	空闲块组 i 的指针
----------------	--------------

空闲块组链的结点结构包括前、后结点指针和本空闲块组的首页框号3个数据域。设当前主存映像图及主存空闲区表 free_area 格式如下：

主存映像	页框号
占用	0
	1
占用	2
	3
	4
	5
占用	6
占用	7
	8
	9
	10
	11
	12
占用	13
	14
	15

块组 号 i	free_area	
	块组位 图指针	空闲块 组指针
0		
1		
2		
3		
4		

问题：

- (1) 根据主存映像图，写出伙伴堆算法的主存空闲区表`free_area`各表项的空闲块组链表和块组位图指向的块组位图内容的数据结构；
- (2) 基于(1)所做的数据结构图，设有长度为3页的内存申请，写出按伙伴堆算法分配页架后的主存映像图和伙伴堆算法数据结构图。

2 在页式存储管理系统中，现有J1、J2和J3共3个作业同驻内存。其中J2有4个页面，被分别装入内存的第3、4、6、8块中。假定页面和存储块的大小均为1024B，主存容量为10KB。

问 题：（1）写出J2的页表；

（2）当J2在CPU上运行时，执行到其地址空间第500号处遇到一条传送指令；MOV 2100, 3100, 请用地址变换图计算MOV指令中两个操作数的物理地址。